



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ
ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛИГИИ”

Дипломна работа

Тема:

ПРОЕКТОРАНЕ И РАЗРАБОТВАНЕ НА СИСТЕМА ЗА ОТЧИТАНЕ И АНАЛИЗ НА ЗАМЪРСЯВАНЕТО НА ВЪЗДУХА

Дипломант: Цветан Христов Ватовски

Образователно-квалификационна степен: бакалавър

Ф.н: 121221159

Специалност: КСИ

Група: 44

Ръководител доц. д-р инж. Иван Станков

София 2025

Съдържание:

Глава 1. ВЪВЕДЕНИЕ И АНАЛИЗ НА ПРОБЛЕМНАТА ОБЛАСТ ...	5
1.1. Значението на локалното измерване и проследяване	6
1.2. Интеграция на измерване, наблюдение и съхранение	7
1.3. Предизвикателства в разработката на подобни системи	8
Глава 2. ЦЕЛ И ЗАДАЧИ НА ДИПЛОМНАТА РАБОТА	10
Глава 3. ПРОЕКТИРАНЕ НА СИСТЕМАТА.....	11
3.1. Основна архитектура.....	11
3.2. Клиент.....	13
3.3. Част Сървър – Контролер	16
3.4. База данни.....	18
3.4.1. Компонент: <i>DatabaseManager</i>	20
3.5. Част Сървър – Графичен интерфейс	22
3.6. Хардуер.....	24
Глава 4. РЕАЛИЗАЦИЯ НА СИСТЕМАТА.....	26
4.1. Използвани технологии.....	26
4.1.1. C++.....	26
4.1.2. SQL	26
4.1.3. Qt.....	26
4.1.4. CMake	26
4.1.5. Shell и BAT скриптове	27
4.1.6. TCP/IP	27
4.1.7. Linux	27
4.1.8. Windows.....	27
4.1.9. BeagleBone Black	27
4.1.10. Термистор MCP970X	28
4.1.11. Gas Sensor MQ-135	28
4.2. Среда на разработка и компилатори	29
4.2.1. Visual Studio Code (VS Code)	29
4.2.2. MSVC (Microsoft Visual C++).....	29
4.2.3. MinGW.....	29
4.2.4. G++ (GCC).....	29
4.3. Реализация на клиент.....	31
4.4. Реализация на сървър	40

4.5. Реализация на хардуерна система	45
Глава 5. РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ	49
ЗАКЛЮЧЕНИЕ	56
ИЗПОЛЗВАНИ ИЗТОЧНИЦИ.....	57
ПРИЛОЖЕНИЕ.....	58

Списък с фигури, използвани в разработката:

Фигура 3.1 – Архитектура на системата

Фигура 3.2 – UML клас диаграма на Клиент

Фигура 3.3 – UML клас диаграма на Контролер

Фигура 3.4 – ER диаграма на базата

Фигура 3.5 – Графичен интерфейс на приложението

Фигура 3.6 – Хардуерни компоненти

Фигура 4.1 – Диаграма на последователностите

Фигура 5.1 – Начално състояние на системата

Фигура 5.2 – Визуализация на един свързан клиент

Фигура 5.3 – Задаване на стойност на интервал и трешхолд на клиент

Фигура 5.4 – Част от текстов файл на клиент, локален за сървъра. Нотификации.

Фигура 5.5 – Част от текстов файл на клиент, локален за сървъра. Поискани данни.

Фигура 5.6 – Визуализирани данни от бутона „Show Data from Client“

Фигура 5.7 – Графики на получените данни в реално време

Фигура 5.8 – Графики на получените данни в реално време. Завишени резултати.

Фигура 5.9 – Визуализация на повече от един свързани клиенти

Фигура 5.10 – Част от текстов файл на клиент, локален за клиента. Преди “Reset Log”.

Фигура 5.11 – Част от текстов файл на клиент, локален за клиента. След “Reset Log”.

Списък с таблици, използвани в разработката:

Таблица 1: **DB Entity CLIENTS table**

Таблица 2: **DB Entity NOTIFICATED_RECORDINGS table**

Таблица 3: **DB Entity REQUESTED_RECORDINGS table**

Таблица 4: **Типове данни на полетата в базата данни**

Таблица 5: **Gas Sensor MQ-135 връзка с Beaglebone Black**

Таблица 6: **Termistor MCP970X връзка с Beaglebone Black**

Таблица 7: **Класификация на стойностите на AQI (Air Quality Index)**

Глава 1. ВЪВЕДЕНИЕ И АНАЛИЗ НА ПРОБЛЕМНАТА ОБЛАСТ

В съвременния свят наблюдаваме все по-осезаемо въздействие на факторите на околната среда върху качеството на живот, здравословното състояние на хората и устойчивостта на градската среда. Проблемите, свързани със замърсяването на въздуха, резките температурни колебания и липсата на достоверна и навременна информация за околната среда, засягат не само големите градове, но и по-малки населени места, където липсва добре изградена инфраструктура за мониторинг и предупреждение.

Особено сериозен е въпросът със замърсяването на въздуха – фактор, който невинаги е видим с просто око, но който има дългосрочни последствия върху човешкото здраве. Според Световната здравна организация, замърсяването на въздуха е един от водещите фактори за развитие на респираторни и сърдечно-съдови заболявания, както и за намалена продължителност на живота. Особено уязвими са деца, възрастни хора и хора с хронични заболявания, които често не разполагат с информация за моментното състояние на въздуха в тяхното обкръжение.

Наличието на климатични промени, индустриално замърсяване, транспортни емисии и дори локални фактори като използване на твърдо гориво в бита, създават условия, в които възниква необходимостта от локализиран, гъвкав и достъпен подход към мониторинг на параметрите на околната среда. Класическите държавни станции за наблюдение на въздуха често са ограничени по брой, разполагат с по-бавно обновяване на данните и не обхващат в детайли всички зони от дадено населено място.

Оттук произтича необходимостта от по-гъвкави решения, които да се интегрират лесно в домашна, учебна или лабораторна среда, да предлагат в реално време информация за качеството на въздуха и температурата, и същевременно да бъдат достъпни както технически, така и финансово.

Развитието на вградени системи, достъпни микроконтролери и сензори, както и широкото използване на интернет технологии, създават предпоставки за изграждане на такива интелигентни системи за мониторинг. Те дават възможност не само за събиране на данни, но и за визуализация, съхранение и анализ, което превръща суровите измервания в реално приложима информация.

Друго важно направление в проблемната област е липсата на лесни за използване потребителски интерфейси, които да позволяват на обикновения потребител – без специализирани технически познания – да наблюдава, анализира и използва събраната информация. Много от съществуващите системи са насочени към специалисти и не предоставят подходящо визуално или функционално ниво на взаимодействие.

1.1. Значението на локалното измерване и проследяване

Значението на локалното измерване и проследяване на параметри на околната среда се основава на факта, че дори в рамките на един и същи град стойностите на замърсяване на въздуха, температурата и други фактори могат да се различават значително в зависимост от множество локални обстоятелства – трафик, индустриално присъствие, наличие на зелени площи, топографски особености и микроклимат. Измерванията, извършвани директно в конкретни помещения, дворове или улици, често показват отклонения спрямо официално публикуваните стойности от централизирани станции, които обикновено предоставят усреднени или обобщени данни. В този контекст локалното събиране на данни се превръща в особено ценно средство за прецизна оценка на заобикалящата среда и взимане на решения, съобразени с реалната ситуация в даден момент и на конкретно място.

Особено значение това придобива в ситуации, при които хората страдат от алергии, хронични дихателни заболявания или други състояния, чувствителни към промени в качеството на въздуха. За тях знанието за моментните нива на замърсяване в дома, училището или работното място може да бъде решаващо за избора на поведение – например проветряване, временно напускане на помещението, използване на пречиствателни устройства или промяна в маршрута на придвижване. Това също така важи в индустриални и лабораторни условия, където се работи с потенциално опасни вещества или в среди с повишен риск – локалният мониторинг тук не само повишава безопасността, но и служи като механизъм за навременна реакция при инциденти.

Системите за локално измерване намират широко приложение и в образователния процес. Чрез включване на ученици и студенти в реални експерименти, при които се събират и анализират данни от тяхната собствена среда, се повишава интересът към природните науки, екологията и технологиите. Това създава възможности за прилагане на интердисциплинарен подход, съчетаващ физика, химия, информатика и здравно образование в една практическа дейност с реална стойност.

Освен в индивидуален и образователен план, локалните измервания са от полза и за създаване на по-точни локализирани прогнози и модели, които отразяват реалното състояние на микросредата. Те позволяват наблюдение на дългосрочни тенденции, откриване на периодични зависимости и установяване на отклонения от нормата, които могат да бъдат сигнал за започване на превантивни действия. Например, ако в даден район системно се наблюдава повишена концентрация на вредни газове в сутрешните часове, могат да се предприемат мерки за ограничаване на трафика или подобряване на вентилацията.

Не на последно място, локалният мониторинг допринася и за повишаване на информираността и личната ангажираност на хората по въпросите на околната среда. Когато индивидите имат достъп до данни, свързани пряко с тяхното ежедневие, те по-лесно осъзнават значението на проблемите, реагират по-отговорно и участват активно в процесите на опазване и подобряване на жизнената среда. В този смисъл локалното измерване не е само технически процес, а и инструмент за изграждане на по-информирано и устойчиво общество.

1.2. Интеграция на измерване, наблюдение и съхранение

Интеграцията на измерване, наблюдение и съхранение на данни е сърцевината на всяка ефективна система за мониторинг на околната среда. За да бъде една такава система пълноценна и надеждна, тя трябва да обединява в себе си няколко взаимосвързани процеса, които функционират координирано и без прекъсвания. Първата и най-основна функция е измерването, което се осъществява чрез подходящо подобрени сензори, отговарящи на конкретните нужди на средата. Тези сензори трябва да бъдат способни да регистрират промени в параметри като температура, влажност, концентрация на вредни газове и други, като при това да осигуряват достатъчна точност и стабилност във времето.

След събирането на суровите данни, те трябва да преминат през етап на обработка, при който се филтрират шумове, коригират се стойности при установени отклонения и се нормализират резултатите така, че да бъдат съпоставими в различни контексти. Това е особено важно при дългосрочно наблюдение или при сравнение на резултати от различни устройства. Необработените сурови данни често са трудни за интерпретация и могат да доведат до погрешни заключения, ако не бъдат представени в подходящ контекст.

Следващият критичен компонент е предаването на информацията към централен елемент – обикновено това е сървърна система, която има за задача да събира данните от различни клиентски устройства. Тук от значение е надеждността на връзката, както и способността на системата да работи с множество източници едновременно. Без стабилна комуникационна структура дори най-прецизните измервания губят своята стойност, тъй като не достигат до потребителя навреме.

Не по-малко важна е визуализацията на събраната информация. Данните, представени в сух числов вид, не са лесно достъпни и разбираеми за крайния потребител. Поради тази причина системата трябва да предлага начини за визуално и интуитивно представяне на измерванията – чрез графики, диаграми или таблични структури, които позволяват проследяване на тенденции, отклонения и сравнения. Ефективната визуализация не само прави системата по-лесна за използване, но и увеличава нейната практическа стойност, тъй като позволява вземане на информирани решения в реално време.

Ако една система не притежава дори само един от гореизброените компоненти, нейното приложение е сериозно ограничено. Например, система, която извършва точно измерване, но не показва данните визуално, ограничава възможността на потребителя да разбере и интерпретира получените стойности. Ето защо пълната интеграция на измерване, обработка, предаване и визуализация е фундаментално изискване за всяко съвременно решение в областта на мониторинга на околната среда.

1.3. Предизвикателства в разработката на подобни системи

Разработката на съвременни системи за мониторинг на околната среда изисква много повече от просто свързване на няколко сензора към компютърна система. Истинските предизвикателства се крият в детайлите, в способността на решението да функционира устойчиво във времето, да бъде полезно на различни типове потребители и да се адаптира към разнообразни сценарии на употреба. На практика, всяка подобна система трябва да отговаря на редица функционални, технически и потребителски изисквания, за да бъде смислена и приложима в реална среда.

На първо място стои въпросът за надеждността и непрекъснатостта на измерванията. Системата трябва да бъде в състояние да функционира денонощно, без сринове, неточности или пропуски в събирането на данни. Това означава устойчив хардуер, добре проектирана логика на работа и защита от ситуации като прекъсване на електрозахранването, срыв на мрежата или отказ на даден компонент. В противен случай, цялата стойност на събраната информация може да бъде компрометирана, а доверието в системата – загубено.

Следващото съществено предизвикателство е свързано с минимизирането на нуждата от поддръжка от страна на потребителя. Идеалната система трябва да бъде „инсталирай и забрави“ – да не изисква непрекъснати проверки, актуализации, рестартиране или технически намеси. Това особено важи в случаите, когато крайните потребители са хора без техническа подготовка или когато системата трябва да работи в отдалечени или труднодостъпни места, където всяка намеса е свързана с допълнителни разходи.

Трето, изключително важно е как данните се представят на крайния потребител. Достъпността на интерфейса и лесната интерпретация на показанията са фактори, които често се подценяват, но на практика решават дали една система ще бъде използвана ефективно. Потребителският интерфейс трябва да бъде интуитивен, визуално ясен и адаптивен спрямо нуждите – както на професионалисти, така и на обикновени потребители, които просто искат да проверят дали качеството на въздуха в техния дом е в норма.

Системата трябва да бъде също така способна да обработва едновременно множество източници на информация. Това включва комуникация с няколко клиентски устройства, всяко от които може да се намира на различна физическа локация и да предава различни типове данни. Координирането на такава мрежа изисква стабилна архитектура, добра синхронизация и правилно разпределение на натоварването, за да не се стигне до забавяния, изгубени данни или блокиране на потока от информация.

Накрая, но не на последно място, стои въпросът за разширяемостта на системата. Една ефективна система не трябва да бъде завършена конструкция, затворена за бъдещи промени, а платформа, която позволява лесно надграждане – например чрез добавяне на нови сензори, нови модули за анализ или допълнителни функционалности за визуализация. В динамичната среда на съвременните технологии, където изискванията постоянно се променят, способността за адаптация е ключов фактор за устойчивост и дългосрочна полезност.

Проектът, който е обект на настоящата дипломна работа, е замислен и реализиран именно с идеята да отговори на тези предизвикателства. Чрез внимателен подбор на архитектурата, логиката на работа и взаимодействието с потребителя, се цели създаването на цялостна, компактна и гъвкава система, която да може да бъде използвана както за практически цели – например в домашна или лабораторна среда, така и за образователни нужди, където учениците и студентите могат да добият реална представа за работа със сензорни технологии, данни и взаимодействие между устройства.

Глава 2. ЦЕЛ И ЗАДАЧИ НА ДИПЛОМНАТА РАБОТА

Основната цел на настоящата дипломна работа е създаването на интелигентна, разширяема и визуално достъпна система за мониторинг на параметри на околната среда, с акцент върху температурата и качеството на въздуха. Системата следва да бъде изградена върху основата на вградена клиентска платформа, централен сървър и графичен потребителски интерфейс, които работят съвместно и в реално време. Целта не се изчерпва само с осигуряване на хардуерна функционалност, а включва интеграция на всички компоненти в една завършена екосистема – от сензорното измерване до визуализацията и съхранението на данните.

За реализиране на тази цел, дипломната работа си поставя няколко основни задачи. На първо място, се провежда анализ на съществуващи решения и технологии с цел проучване на съвременните методи за измерване на параметри на околната среда и архитектурите, използвани при подобни системи. След това се осъществява избор на хардуерна платформа, включваща подходящи вградени устройства и сензори за измерване на температура и качество на въздуха. В конкретния случай се използва платка от тип BeagleBone Black^[5], заедно с термистор и газов сензор. На следващ етап се разработва клиентски софтуер, отговорен за събирането на данни от сензорите, тяхната обработка и предаването им към сървър в реално време чрез комуникация по TCP/IP^[11] протокол.

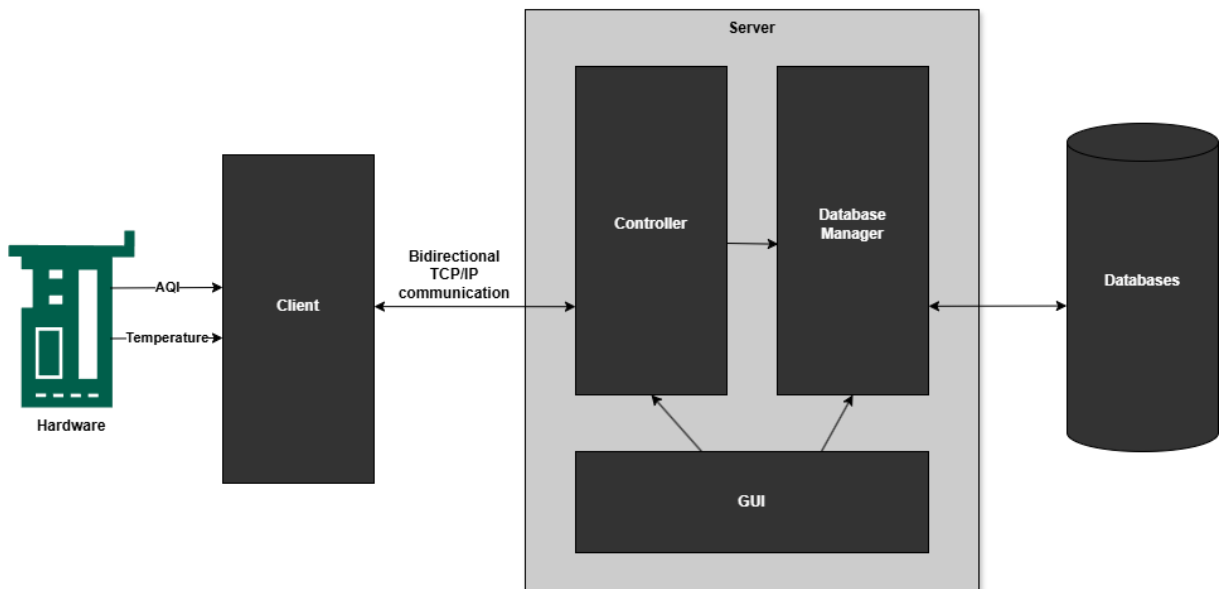
Паралелно с това се изгражда сървърен модул, способен да приема информация от множество клиентски устройства, да я обработва и да извършва контролни действия по заявка от страна на потребителя. За осигуряване на удобство при работа и достъпност на информацията се създава графичен потребителски интерфейс под формата на десктоп приложение, което предоставя визуално представяне на данните, възможност за избор на активен клиент, изпращане на команди и наблюдение чрез динамични диаграми. Особено внимание се отделя и на съхранението на данни, като се реализира функционалност за паралелно записване както в текстови файлове, така и в база данни.

След завършване на основната разработка се провеждат тестове, симулации и валидиране на системата. Това включва изпитване на поведението ѝ при реални измервания, симулации при екстремни стойности и проследяване на реакцията в различни условия. Накрая се извършва систематизиране и анализ на постигнатите резултати, оценка на надеждността и приложимостта на системата в реална среда.

Успешното изпълнение на всички тези задачи не само демонстрира техническа компетентност и умения за интеграция на различни елементи в едно завършено решение, но и създава основа за бъдещо разширяване на проекта. Това може да включва добавяне на нови типове сензори, осигуряване на достъп чрез мобилни устройства или автоматично известяване при превишаване на критични стойности. По този начин дипломната работа се превръща в завършен и практически приложим продукт, който може да намери реализация както в ежедневна, така и в обучителна среда.

Глава 3. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Фигура 3.1 изобразява архитектура на системата:



Фигура 3.1 – Архитектура на системата

3.1. Основна архитектура

Представената архитектура (Фигура 3.1) описва цялостната система за мониторинг на параметри от околната среда – температура и качество на въздуха (AQI^[7]). Системата е разпределена и се състои от два основни участника: *Клиент* (вградено устройство със сензори) и *Сървър* (настолно приложение с графичен интерфейс и бази данни). Всеки компонент има ясно дефинирана роля и комуникацията между тях е организирана така, че да се осигури надеждно измерване, обработка, визуализация и съхранение на данните.

Процесите в системата, са както следва:

- Процес на измерване и изпращане
- Получаване и управление на данните в сървъра
- Обработка и съхранение
- Визуализация и взаимодействие с потребителя

Всичко започва от вграденото устройство (клиент), което е снабдено със сензори за температура и качество на въздуха. Устройството периодично извършва измервания,

като събира стойностите от околната среда и ги подготвя за изпращане към сървъра. Измерванията се изпращат през мрежова връзка по определен TCP/IP^[11] протокол.

На сървърната страна постъпилите данни се обработват от *контролния модул (Controller)*, който има задачата да приема връзки от множество клиенти едновременно, да различава отделните им идентификатори и да координира изпращането и получаването на съобщения. Контролерът действа като посредник между клиентите и останалата част от сървъра, осигурявайки правилно маршрутизиране на данните.

Веднъж приети, данните се препращат към *модула за управление на базата данни (Database Manager)*, който се грижи за тяхното съхранение както на файлово, така и на релационно ниво. Освен съхранението, тук се извършват и допълнителни действия като филтрация, архивиране и извличане при нужда.

Графичният потребителски интерфейс (GUI) предоставя на оператора лесен достъп до всички функционалности. През интерфейса може:

- да се наблюдава в реално време кои клиенти са свързани,
- да се изпращат команди към конкретен клиент (напр. задаване на интервал за измерване),
- да се визуализират стойности под формата на диаграми,
- да се разглеждат запазените данни от базата за избран клиент.

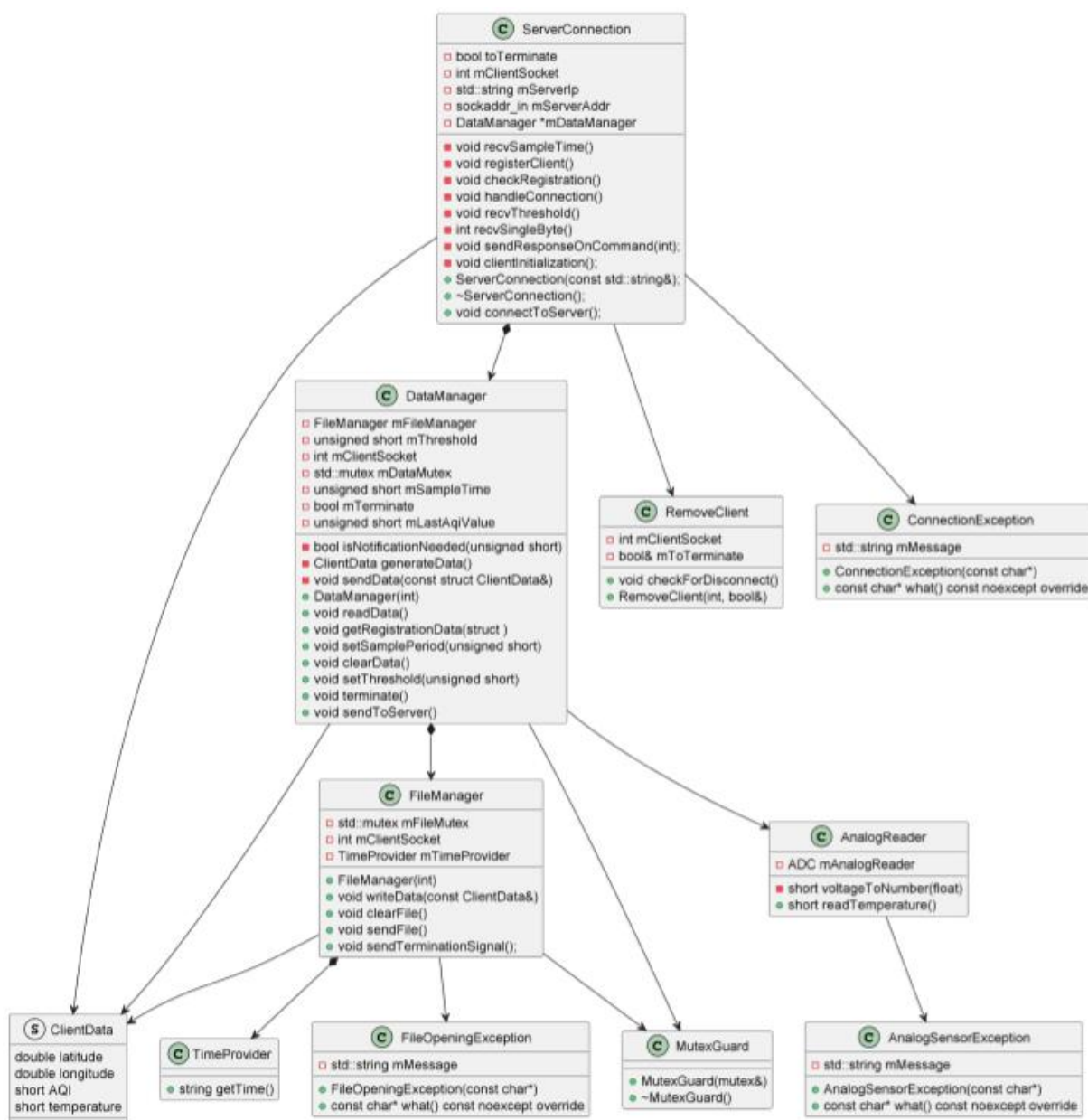
GUI не работи самостоятелно, а комуникира както с контролера (за изпращане на команди и показване на текущ статус), така и с мениджъра на бази данни (за зареждане и показване на данните под формата на диаграми).

Обобщение на потока

1. *Клиент* измерва температура и AQI и изпраща до сървъра.
2. *Контролерът* приема данните и ги разпределя.
3. *Мениджърът на база данни* съхранява полученото в базата.
4. *GUI* дава възможност на потребителя да наблюдава и управлява процеса.

3.2. Клиент

Фигура 3.2 изобразява UML клас диаграма на *Клиента*:



Фигура 3.2 – UML клас диаграма на *Клиент*

Клиентът (Фигура 3.2) е вграден модул, който събира данни от околната среда, обработва ги локално и ги изпраща към сървър за централизирано съхранение и визуализация. Клиентът установява връзка със сървъра чрез мрежа, измерва температура и стойности на замърсяване на въздуха (AQI), оценява дали стойностите изискват известяване, и при необходимост изпраща структурирани данни към сървъра. Управлението на процеса е разделено на няколко основни модула, всеки със свои отговорности.

ServerConnection представлява основния комуникационен модул в клиентската част на системата. Неговата роля е да инициализира и установи TCP^[11] връзка със сървъра, да регистрира клиента, както и да получава и обработва команди от сървъра, свързани с конфигурационни параметри като интервал за измерване и праг за известяване. Този клас също така отговаря за изпращането на отговори при поискване, както и за стартиране или прекратяване на работата на клиента чрез флаг за прекъсване *m_toTerminate*. Всички данни, постъпили по мрежата, се препращат към класа *DataManager*, който реализира логиката за тяхната обработка.

DataManager е ядрото на локалната обработка в клиента. Той отговаря за периодичното събиране на данни чрез сензори и извършва проверка дали дадено измерване надвишава предварително зададен праг чрез метода *isNotificationNeeded*. Ако има отклонение, се инициира изпращане на данните към сървъра чрез *sendToServer*. Освен това, *DataManager* поддържа текущите стойности на прага и интервала за измерване, които могат да бъдат задавани от сървъра. Данните, независимо дали се изпращат или не, се записват локално във файл с помощта на класа *FileManager*. За осигуряване на безопасна работа между конкурентни потоци, *DataManager* използва синхронизационен механизъм чрез *std::mutex*.

ClientData е структура, използвана за съхранение на измерените данни в конкретен момент. В нея се съдържат географските координати *latitude* и *longitude*, които са фиксирани за конкретното устройство, както и двете основни измерени стойности – температура и индекс на качество на въздуха (AQI). Тази структура се използва както при изпращане на данни към сървъра, така и при локалното им записване.

FileManager отговаря за локалното съхранение на данни във файлова система. Всеки запис, който бъде генериран от *DataManager*, може да бъде съхранен чрез този модул. *FileManager* разполага с механизъм за защита на файлови операции чрез *m_fileMutex*, и използва помощния клас *TimeProvider*, за да маркира данните с текущо време. Освен стандартния запис на данни, той поддържа и изпращане на терминален сигнал при спиране на програмата, както и функции за изчистване или затваряне на файла.

TimeProvider е малък спомагателен клас, чиято основна задача е да предоставя текущата времева стойност във формат, подходящ за логване на измерванията. Той се използва от *FileManager*, но може да бъде приложен и в други контексти, когато се изисква времева маркировка.

AnalogReader е отговорен за директната комуникация с хардуерните сензори на устройството. Неговата основна роля е да прочете аналоговите стойности, постъпващи от сензорите за температура и газове, и да ги трансформира до цифрови стойности чрез метод като *voltageToNumber*. Осигурена е и защита от грешки при работа с хардуера чрез клас *AnalogSensorException*, който сигнализира за проблеми при четене.

RemoveClient е компонент, чиято функция е да следи връзката със сървъра и да отчита моментите, когато тя е прекъсната. Той работи с идентификатор на клиента и задава флаг *m_toTerminate*, ако бъде установено, че сървърът не е достъпен. Това осигурява контролирано и сигурно прекратяване на процесите в клиента.

FileOpeningException и *AnalogSensorException* са класове, използвани за обработка на изключения при възникване на грешки, съответно при отваряне на файлове и при четене от сензори. Те осигуряват ясен и структуриран начин за диагностика на проблеми в процеса на работа. Също така, *MutexGuard* е помощен клас, който гарантира безопасен достъп до критични участъци от кода чрез автоматично заключване и отключване на мютекс при създаване и унищожаване на обекти от този тип.

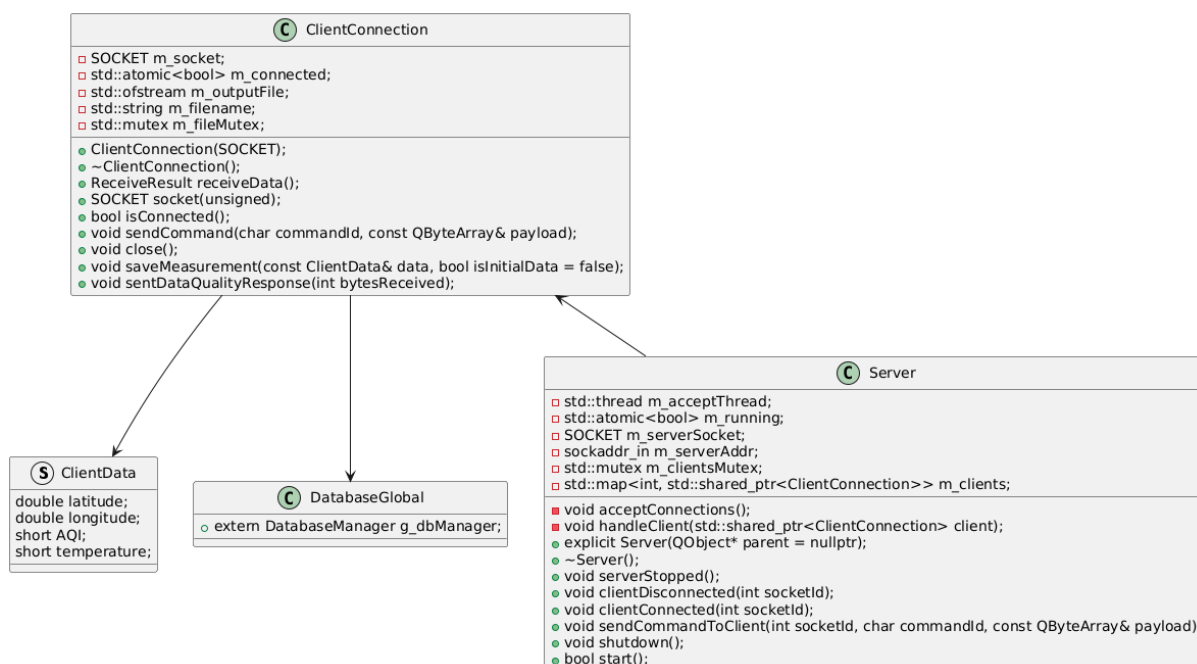
Тази организация на клиентската страна осигурява гъвкавост, устойчивост на грешки и възможност за лесна интеграция със сървърната част от системата.

Типичен цикъл на работа

1. *ServerConnection* установява връзка със сървъра и инициализира *DataManager*.
2. *DataManager* периодично генерира нови данни чрез *AnalogReader*.
3. Ако се налага известяване (над прага), данните се изпращат към сървъра.
4. Успоредно с това, данните се записват локално чрез *FileManager*.
5. В случай на прекъсване, *RemoveClient* прекратява процеса и уведомява сървъра.

3.3. Част Сървър – Контролер

Фигура 3.3 изобразява UML клас диаграма на *Контролера*:



Фигура 3.3 – UML клас диаграма на *Контролер*

Задачата на *контролера* е да управлява комуникацията с клиентите, да приема и разпределя данните, както и да поддържа синхрон между мрежовите връзки, графичния интерфейс и базата данни. В тази архитектура контролерът е реализиран чрез два основни класа: *Server* и *ClientConnection*, както и чрез допълнителни помощни структури като *ClientData* и *DatabaseGlobal*.

Класът *Server* изпълнява ролята на централен комуникационен координатор, който слуша за нови клиентски връзки, приема ги и създава обекти *ClientConnection*, чрез които осъществява индивидуално взаимодействие с всеки клиент. При стартиране чрез метода *start()* се активира нишка (*m_acceptThread*), която изпълнява метода *acceptConnections()*. Тази нишка приема новите клиентски сокеи и ги регистрира в асоциативна структура *m_clients*, която съхранява указатели към съответните обекти *ClientConnection*. Методът *handleClient()* се извиква за всеки приет клиент и отговаря за неговото управление в отделна нишка. Този метод следи дали клиентът е активен, приема данни от него и подава съответната информация към други модули от системата. При отпадане на връзката се извиква *clientDisconnected()*, а при успешно свързване – *clientConnected()*.

Сървърът също така поддържа функционалност за изпращане на команди към даден клиент чрез метода *sendCommandToClient()*. Той приема уникалния идентификатор на клиента (*socket*), команден идентификатор (напр. „задай интервал“, „изтегли лог“ и др.) и съпровождащите данни в *QByteArray*. Методът *shutdown()*

извършва контролирано спиране на сървъра – прекратява нишките, затваря сокета и известява останалите компоненти чрез сигнала *serverStopped()*.

Класът *ClientConnection* представлява абстракция на свързан клиент. Той енкапсулира целия процес на приемане на данни от клиента, обработката им, както и възможността за връщане на команди. Обектът се създава със специфичен сокет, чрез който комуникацията се поддържа активно. Основният флаг *m_connected* показва дали клиентът е все още в активно състояние.

Методът *receiveData()* управлява приемането на данни от клиента, които включват стойности на температура и AQI^[7]. След успешен прием, данните се предават към метода *saveMeasurement()*, който ги записва както във файл, така и в база данни чрез глобалната променлива *g_dbManager*, дефинирана в *DatabaseGlobal*. Методът *sendCommand()* се използва от сървъра, когато е необходимо да се изпрати обратно команда към клиента, напр. промяна на настройка или изтриване на данни. Допълнителната функционалност включва метод *sentDataQualityResponse()*, който може да се използва за връщане на информация относно коректността на получените данни. За сигурен достъп до изходния файл при запис се използва мютекс *m_fileMutex*, а самият запис се извършва във файл с име, съхранявано в *m_filename*.

ClientData е проста структура, която съдържа четирите ключови стойности, които се обменят между клиент и сървър: географски координати (latitude, longitude), температура (temperature) и индекс на замърсяване на въздуха (aqi). Тя служи като универсален контейнер за предаване на измервания в системата.

Файлът *DatabaseGlobal* предоставя глобален достъп до единен обект *DatabaseManager*, чрез който сървърът осъществява операции с базата данни – като съхранение, извличане или изчистване на данни. Този подход позволява независимите компоненти *ClientConnection* и *Server* да взаимодействат с базата чрез споделен интерфейс.

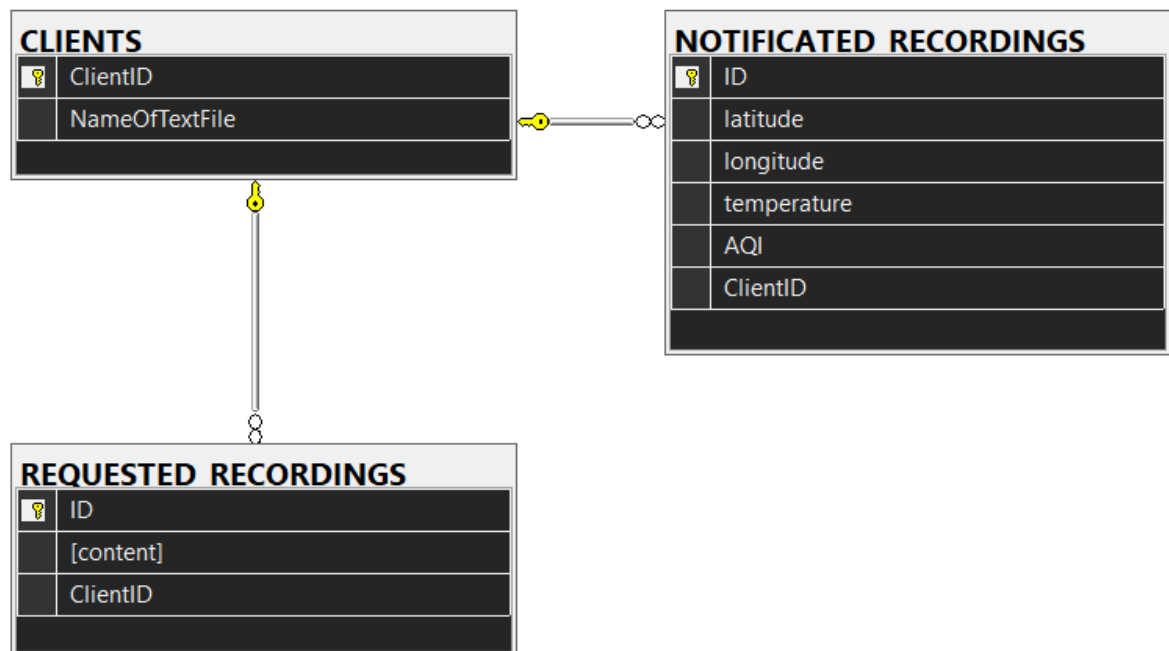
Обобщение на процеса

1. Сървърът стартира и започва да слуша за нови връзки.
2. При свързване на клиент се създава *ClientConnection* и започва комуникацията.
3. Клиентът изпраща измерени стойности, които се получават и записват от *ClientConnection*.
4. Сървърът визуализира информацията и/или я съхранява в базата чрез глобалния мениджър.
5. При нужда, GUI частта може да изпрати команда към клиент чрез *sendCommandToClient*.
6. При отпадане на клиент или при изрично спиране, ресурсите се освобождават и сървърът се изчиства.

Тази структура осигурява мащабируемост, поддръжка на множество клиенти едновременно и ясно разграничение между комуникация, обработка и съхранение на данните.

3.4. База данни

Фигура 3.4 изобразява ER диаграмата на базата:



Фигура 3.4 – ER диаграма на базата

ER (Entity-Relationship) диаграмата (Фигура 3.4) представя структурата на базата данни, използвана от сървърната част на системата. Тя описва трите основни таблици и релациите между тях, като всяка таблица е свързана с конкретен аспект от процеса на събиране, съхранение и достъп до данни от клиентските устройства.

Таблица 1: DB Entity CLIENTS table

Поле	Тип на данните	Описание
ClientID	INT (PK)	Уникален идентификатор на клиента. Използва се като първичен ключ и за външни връзки в други таблици.
NameOfTextFile	VARCHAR или NVARCHAR	Име на локалния текстов файл, асоцииран с клиента. Това име се използва за съхранение на данни и извличане при нужда.

Забележка: Тази таблица (Таблица 1) е свързана с NOTIFICATED_RECORDINGS и REQUESTED_RECORDINGS чрез външни ключове към ClientID.

Таблица 2: **DB Entity NOTIFICATED_RECORDINGS table**

Поле	Тип на данните	Описание
ID	INT (PK)	Уникален идентификатор на всяка записана стойност.
latitude	FLOAT или DOUBLE	Географска ширина на устройството.
longitude	FLOAT или DOUBLE	Географска дължина на устройството.
temperature	SMALLINT или INT	Измерена температура от клиента.
AQI	SMALLINT или INT	Измерен индекс на качеството на въздуха (Air Quality Index).
ClientID	INT (FK)	Външен ключ, сочещ към таблицата CLIENTS, чрез който се идентифицира от кой клиент идва даденият запис.

Релация: Много към един (Many-to-One) – множество записи могат да са асоциирани с един клиент.

Таблица 3: **DB Entity REQUESTED_RECORDINGS table**

Поле	Тип на данните	Описание
ID	INT (PK)	Уникален идентификатор на заявения запис.
content	VARCHAR (MAX) или TEXT	Съдържание на записа – може да бъде текстов формат на лог, команда или друга заявена от клиента информация.
ClientID	INT (FK)	Външен ключ към CLIENTS, указващ кой клиент е инициирал или е свързан със заявката.

Релация: Много към един (Many-to-One) – множество заявени данни могат да бъдат свързани с един клиент.

Релации между таблиците:

- CLIENTS (1) → (∞) NOTIFICATED_RECORDINGS
Всеки клиент може да има множество измерени стойности, които се запазват само ако надвишават зададен праг.
- CLIENTS (1) → (∞) REQUESTED_RECORDINGS
Всеки клиент може да направи множество заявки за данни, които се съхраняват в тази таблица.

Таблица 4: Типове данни на полетата в базата данни

Поле	Тип
ClientID, ID	INT IDENTITY(1,1) PRIMARY KEY
NameOfTextFile	NVARCHAR(255)
latitude, longitude	FLOAT
temperature, AQI	SMALLINT
content	NVARCHAR(MAX)

Тази структура е подходяща за мониторинг система, в която:

- CLIENTS съхранява статични данни за устройствата.
- NOTIFICATED_RECORDINGS пази критични измервания (при преминаване на праг).
- REQUESTED_RECORDINGS съдържа допълнителна информация по заявка на потребителя или администратора.

Системата осигурява ясна проследимост, мащабируемост и удобство за анализ и визуализация.

3.4.1. Компонент: *DatabaseManager*

DatabaseManager представлява централен клас в сървърната архитектура, отговарящ за цялостното взаимодействие с релационната база данни. Той е реализиран като самостоятелен модул и осигурява интерфейс за запис, четене и изчистване на данни, без останалите компоненти да се налага да познават детайлите на SQL заявките или структурата на таблиците.

DatabaseManager изпълнява ролята на централен модул за управление на данните, получени от клиентските устройства, в сървърната архитектура. Една от основните му отговорности е записът на измервания от клиенти. Когато клиент изпрати измерване, съдържащо температура, индекс на качеството на въздуха (AQI^[7]) и географски координати, този модул отговаря за съхраняването на тези стойности в таблицата *NOTIFICATED_RECORDINGS*. Методът, използван за тази цел, приема идентификатор на клиента и структура *ClientData*, след което чрез SQL INSERT заявка вписва данните, като ги асоциира с правилния *ClientID*.

Друга ключова функция е регистрирането на нови клиенти. При всяко ново свързване, *DatabaseManager* проверява дали съответният клиент вече е регистриран в таблицата *CLIENTS*. Ако не е, той създава нов запис със съответния *ClientID* и добавя името на асоциирания текстов файл, който служи за локално съхранение. По този начин се гарантира консистентна връзка между всеки клиент и неговите данни в базата.

Освен автоматичните записи, системата поддържа и възможност за ръчни заявки, които се въвеждат чрез потребителския интерфейс. Тези данни се обработват от

DatabaseManager и се записват в таблицата *REQUESTED_RECORDINGS*. Методът, който изпълнява тази задача, приема текстово съдържание и съответния клиентски идентификатор. Това улеснява съхраняването на заявки за логове, отчети и справки, които не са резултат от автоматични измервания, но все пак имат значение за анализа.

DatabaseManager предлага и функционалност за извличане на исторически данни. Графичният интерфейс на сървъра използва този модул за достъп до съществуващите записи. Класът предоставя методи за извличане както на последните N измервания за даден клиент, така и на всички натрупани данни от момента на регистрацията му. Извлечените стойности се връщат под формата на вектор от стрингове или структури и се използват за визуализация в интерфейса или за допълнителен анализ.

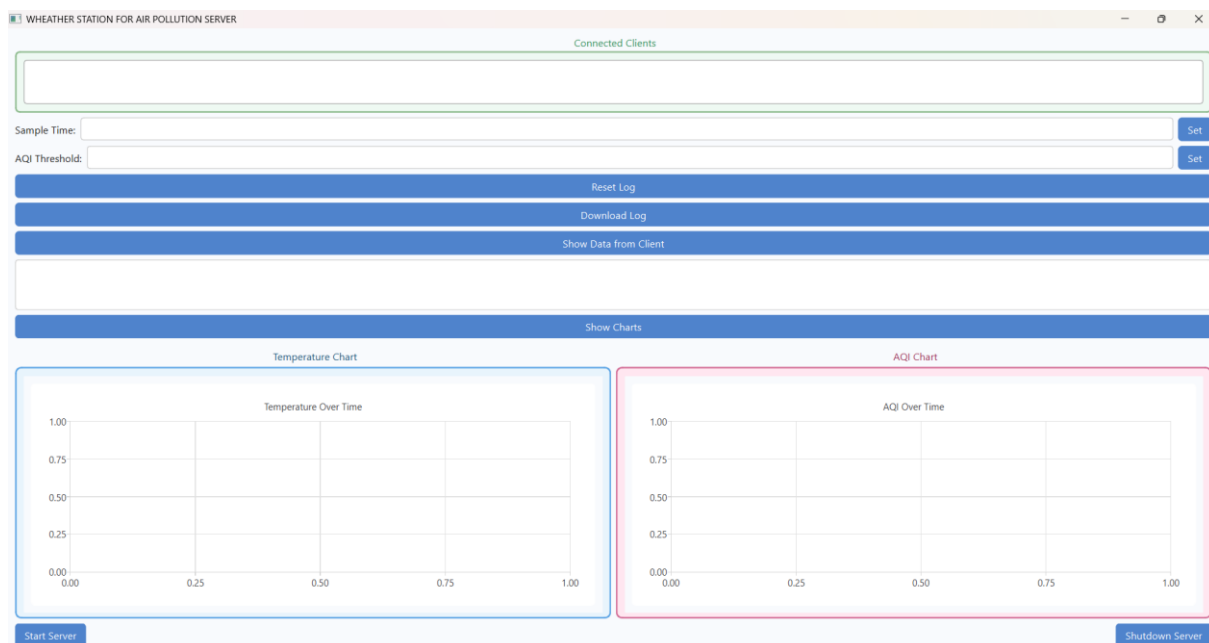
При рестартиране на сървъра или при изричното му изключване, *DatabaseManager* има възможност да изчисти всички временни записи от базата. Това става чрез изпълнение на DELETE заявки към съответните таблици, с което се осигурява чисто начално състояние на базата при следващо стартиране. Така се предотвратява натрупването на стари или нерелевантни данни и се запазва производителността на системата.

Свързаност с други компоненти:

DatabaseManager се използва глобално чрез обект *g_dbManager*, който е дефиниран в *DatabaseGlobal.hpp*. Това улеснява достъпа до базата от различни части на сървъра (напр. *ClientConnection*, *Server*, *MainWindow*) без нужда от допълнителна инициализация.

3.5. Част Сървър – Графичен интерфейс

Фигура 3.5 представя визията на сървърната част, дадена от GUI:



Фигура 3.5 – Графичен интерфейс на приложението

Графичният потребителски интерфейс (GUI - Фигура 3.5) на сървърното приложение представлява интуитивна и функционална среда, чрез която операторът може да управлява, наблюдава и взаимодейства с цялостната система за мониторинг на температура и качество на въздуха. Интерфейсът е реализиран с помощта на Qt 6^[3] и комбинира различни визуални компоненти за контрол, визуализация и отчетност.

В горната част на прозореца се намира поле, обозначено с *Connected Clients*. То представлява списък с всички в момента свързани клиентски устройства, идентифицирани чрез уникален socket ID. При всяко ново свързване или прекъсване на клиент, списъкът се обновява автоматично, позволявайки на потребителя да избере активен клиент, с когото да работи.

Под списъка с клиенти се намират две текстови полета – *Sample Time* и *AQI Threshold*. Те служат за въвеждане на стойности, които се изпращат като команди към избрания клиент. *Sample Time* определя интервала, през който устройството ще извършва ново измерване, а *AQI Threshold* указва прагова стойност за известяване при превишаване на допустимите нива на замърсяване. Всяко от полетата има бутон *Set*, чрез който въведената стойност се подава към сървъра и съответно препраща до конкретния клиент чрез мрежова команда.

Следва група от три основни бутона с разширена функционалност. *Reset Log* изпраща команда до избрания клиент за изтриване на локално съхранените измервания. *Download Log* стартира процес на изтегляне на натрупаните данни от клиента към

сървър. *Show Data from Client* позволява на оператора да види всички записи, съхранени в базата данни за конкретния клиент. След избиране на клиента и натискане на бутона, данните се визуализират в текстовото поле под него, като всяка стойност е изведена на отделен ред.

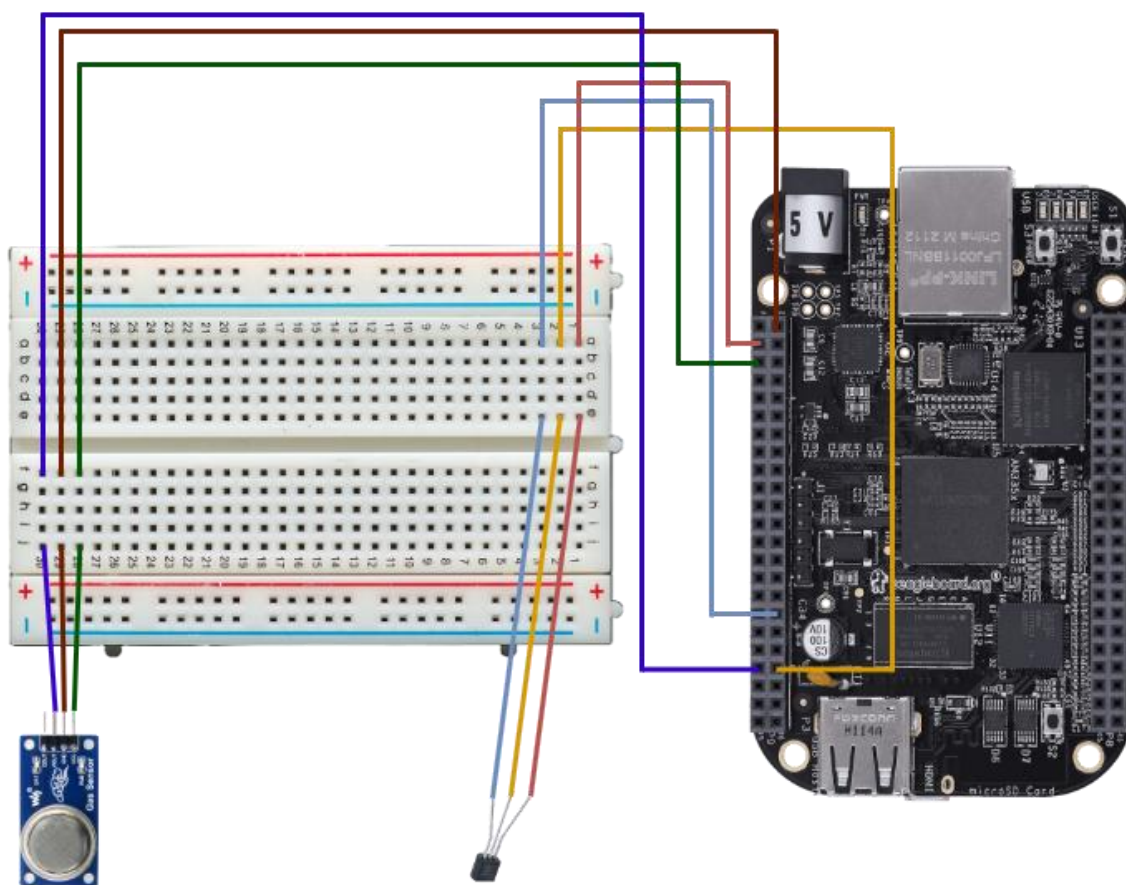
Централната част на интерфейса е посветена на графична визуализация. Бутонът *Show Charts* стартира процес по обновяване на двете динамични диаграми – *Temperature Chart* и *AQI Chart*. Те представят последователно стойности на температурата и качеството на въздуха за избрания клиент. Диаграмите са изградени с помощта на *QLineSeries* и се обновяват автоматично на всеки няколко секунди чрез вътрешен таймер. Те показват последните 50 измервания, като осите се мащабират динамично според стойностите.

В долната част на прозореца се намират два бутона за управление на сървър. *Start Server* стартира фоновата нишка, в която сървърът започва да приема клиентски връзки и да разпределя комуникационните потоци. При успешен старт всички други функции стават активни. Бутонът *Shutdown Server* прекратява работата на сървър, затваря връзките с клиентите, почиства данните от базата и деактивира останалите контроли.

В обобщение, графичният интерфейс служи като удобна точка за наблюдение и управление на цялата система. Той осигурява визуален достъп до клиентските данни, позволява изпращане на команди, следи състоянието на свързаните устройства и визуализира промените в реално време чрез графики и текст. Интерфейсът е проектиран да бъде лесен за употреба, дори от оператори без задълбочен технически опит.

3.6. Хардуер

Фигура 3.6 представя хардуерните компоненти:



Фигура 3.6 – Хардуерни компоненти

Представената схема (Фигура 3.6) изобразява хардуерната конфигурация на клиентското устройство в системата за мониторинг на температура и качество на въздуха. Тя демонстрира начина, по който отделните сензори и компоненти са свързани към микроконтролера *BeagleBone Black*^[5], използван като основна изчислителна платформа на клиента. С помощта на тестова платка (*breadboard*) се осъществява удобна и прецизна организация на електрическите връзки между елементите, без нужда от постоянно спояване.

В долната лява част се намира *газовият сензор*, който измерва замърсеността на въздуха. Неговите изходи са свързани към захранване (5V и GND), както и към един от аналоговите входове на *BeagleBone Black*^[5], който позволява прочитане на стойности на концентрацията на газове (AQI).

До него се намира *температурният сензор*, свързан също чрез три проводника – захранване, земя и аналогов сигнал. Аналоговият изход на този сензор е свързан към различен аналогов вход на микроконтролера, което позволява паралелно измерване на температурата, независимо от AQI модула.

Проводниците, изобразени в различни цветове, показват ясна и организирана връзка между всеки сензор и конкретен пин на платката. Използвани са няколко аналогови входа (AIN), както и стандартни GPIO пинове за управление или допълнителни функции, които могат да бъдат активирани при необходимост.

Breadboard-ът играе роля на разпределителен възел – към него са свързани всички компоненти, а връзките между пиновете и модулите се реализират чрез кабели с твърда изолация. Това осигурява стабилна, но лесно променяема връзка по време на прототипиране и тестове.

От дясната страна се намира самата платка *BeagleBone Black*, към която водят всички връзки. Тя осигурява необходимото захранване, както и интерфейсите за комуникация и обработка. Тя е в състояние да събира, анализира и изпраща данните в реално време към сървъра чрез клиентския софтуер, който работи на нея.

Цялостната схема демонстрира балансирано решение, съчетаващо сензори, аналогови входове, захранване и комуникация, обединени в една физическа конфигурация, готова за разполагане в реална среда. Макар да не навлизаме тук в детайли за конкретните сензори, стойности или типове сигнали, изображението и описаната логика ясно показват функционалното разделение и потока от данни от физическия свят до цифровата обработка.

Глава 4. РЕАЛИЗАЦИЯ НА СИСТЕМАТА

4.1. Използвани технологии

4.1.1. C++

C++^[1] е обектно-ориентиран език за програмиране от системно ниво, известен със своята ефективност, бързодействие и пряк достъп до хардуера. В настоящия проект C++ е използван както за сървърната, така и за клиентската част на приложението. С него се реализират основните комуникационни механизми, обработката на сензорни данни, записът във файлове и бази, както и основната логика на програмата. Използван е стандартът C++17^[4], който предлага модерни функционалности като `std::shared_ptr`^[14], `std::thread`^[14], и подобрен синтаксис, което позволява по-сигурна и четима реализация.

4.1.2. MS SQL (Microsoft SQL Server)

MS SQL Server^[15] е система за управление на релационни бази данни, разработена от Microsoft. В проекта се използва за централизирано съхранение на получените измервания, регистрация на клиентите и обработка на потребителски заявки от графичния интерфейс. Използването на MS SQL Server осигурява надеждност, сигурност и лесна поддръжка на данните. Тестовата и развойната среда използват SQL Server Express Edition 2019^[15], който е безплатен и напълно достатъчен за нуждите на системата.

4.1.3. Qt

Qt^[3] е широко използван фреймуърк за създаване на кросплатформени графични интерфейси и приложения. Проектът използва Qt 6, по-конкретно неговите компоненти Qt Widgets^[3] и Qt Charts^[3]. С Qt е реализиран GUI на сървър, който осигурява визуален контрол върху данните, избора на клиенти, изпращането на команди и визуализацията чрез графики. Qt е предпочетен заради своята стабилност, богат набор от функции и интеграция с C++^[1].

4.1.4. CMake

CMake^[10] е инструмент за автоматизирано конфигуриране и генериране на build системи за C++ проекти. Той позволява създаване на платформено-независими конфигурации и е използван за изграждането на целия проект – както клиента (Linux), така и сървър (Windows). Чрез CMake^[10] се дефинират зависимости, включват се директории, и се активира автоматичната компилация на Qt файлове (AUTOMOC, AUTOUIIC, AUTORCC). Използваната версия е 3.16 или по-висока.

4.1.5. Shell и BAT скриптове

Shell (за Linux)^[12] и BAT (за Windows)^[12] скриптове са използвани за автоматизиране на процесите по компилация, стартиране и разпределяне на проектните файлове. Скриптовите се грижат за създаване на build директории, извикване на CMake^[10] и копиране на нужните библиотеки. Това улеснява разработката, ускорява тестовите и намалява възможността от грешки при ръчно изграждане.

4.1.6. TCP/IP

TCP/IP^[11] (Transmission Control Protocol / Internet Protocol) е основен мрежов протокол за комуникация в интернет и локални мрежи. Целият проект разчита на TCP/IP за връзка между клиентите и сървъра. Клиентът установява TCP^[11] сесия със сървъра, по която се предават команди и измервания. TCP е избран заради своята надеждност, гарантирана доставка и подреденост на съобщенията – нещо критично за работа с измервателни данни.

4.1.7. Linux

Linux е използван като операционна система за клиентската част на системата. Клиентският код се стартира на вграденото устройство BeagleBone Black, което използва Linux дистрибуция като Debian или подобна. Изборът на Linux се обуславя от стабилността, лекотата за конфигуриране, богатата поддръжка на хардуерни интерфейси (GPIO, ADC), и отворения код, което го прави подходящ за вградени решения.

4.1.8. Windows

Windows е операционната система, върху която е реализиран и стартиран сървърният компонент. Това включва графичния потребителски интерфейс, свързаността с MS SQL Server, и по-лесната визуална работа с диаграми, текст и администрация на данни. Изборът на Windows за сървъра позволява и лесно използване на Visual Studio, Qt Designer и други инструменти, които подобряват продуктивността.

4.1.9. BeagleBone Black

BeagleBone Black^[5] е вграден хардуерен микрокомпютър, който служи като клиент в настоящата система. Той има достатъчно изчислителна мощност, поддръжка на аналогови входове и работи под Linux. Благодарение на компактните си размери, ниска консумация и поддръжка на различни сензори, той е идеален избор за реализация на клиент, който събира и изпраща данни от реална среда.

4.1.10. Термистор MCP970X

MCP970X^[9] е тип термисторен аналогов сензор, използван за измерване на температурата. Той предоставя аналогов сигнал, пропорционален на измерената температура, който се прочита от BeagleBone Black чрез ADC интерфейс. Предимствата му включват ниска цена, лесна употреба и добра точност за вградени приложения.

4.1.11. Gas Sensor MQ-135

MQ-135^[8] е аналогов сензор за качество на въздуха, чувствителен към различни вредни газове и замърсители. Използва се за измерване на стойности, свързани с замърсяване (AQI), и предава аналогов сигнал към микроконтролера. MQ-135^[8] е широко използван в хоби и професионални проекти, заради своята достъпност и способност да засича множество газове, включително амоняк, бензен, алкохоли и дим.

Всички изброени технологии работят съвместно в една интегрирана система, където клиентите събират данни от физическата среда и ги изпращат към сървър, който визуализира, съхранява и управлява потока от информация. Изборът на тези инструменти и платформи е направен с оглед стабилност, ефективност, достъпност и съвместимост между отделните компоненти на системата.

4.2. среда на разработка и компилатори

Средата за разработка и използваните компилатори играят ключова роля в изграждането, поддръжката и преноса на проекта между различните платформи. Настоящият проект е реализиран с акцент върху кросплатформена съвместимост, което изисква внимателен избор на инструменти както за Windows, така и за Linux базирани среди.

4.2.1. Visual Studio Code (VS Code)

Visual Studio Code е лека, модулна и широко използвана среда за разработка (IDE), създадена от Microsoft. Избрана е заради своята кросплатформеност, богатата екосистема от разширения и отлична поддръжка за C++ чрез разширения като C/C++ for Visual Studio Code (by Microsoft), CMake Tools^[10] и CMake Language Support^[10]. VS Code е използвана както под Windows, така и под Linux, което осигурява еднакъв работен процес на двете платформи. Средата предоставя възможности за автоматично компилиране, дебъгване, навигация в кода, цвятова маркировка, IntelliSense, и управление на проекти чрез Git.

4.2.2. MSVC (Microsoft Visual C++) – за Windows

Под Windows, проектът се компилира чрез компилатора MSVC, който е част от Visual Studio Build Tools. Използваната версия е *MSVC 19.39.33521.0*, съвместима със стандарта C++17^[4]. Този компилатор предлага отлична оптимизация за изпълнение под Windows, пълна съвместимост с Qt 6 и стабилна интеграция с CMake^[10] чрез генератор "*NMake Makefiles*". Благодарение на MSVC, изграждането на графичния интерфейс, комуникационния сървър и базовата логика на приложението се осъществява с висока производителност и стабилност.

4.2.3. MinGW – за Unix-базирани среди (Linux и BeagleBone Black)

MinGW (Minimalist GNU for Windows) е използван при необходимост за компилация под Unix-подобни среди в симулации и тестове. Въпреки това, основният компилатор за клиента (на BeagleBone Black) е g++, който идва като част от GNU Compiler Collection (GCC). При необходимост от компилиране на проекта и под Windows за Linux-базирана архитектура, MinGW предлага добър начин за това чрез крос-компиляция.

4.2.4. G++ (GCC) – за BeagleBone Black

На самото вградено устройство *BeagleBone Black*, клиентският софтуер се компилира локално чрез компилатора g++, версия **8.3.0**, който е част от стандартната

инсталация на Debian Linux^[6]. BeagleBone Black използва Debian дистрибуция, адаптирана за ARM процесор, с пълна поддръжка за компилиране на C++ код, достъп до аналогови пинове, GPIO, I2C и други интерфейси. Компилацията се извършва директно върху устройството или чрез SSH и build скрипт, базиран на CMake. G++ предлага стабилна, добре поддържана и широко разпространена среда за вградени приложения.

Изборът на Visual Studio Code като единна IDE позволява работа с един и същ интерфейс на различни операционни системи. MSVC е предпочетен заради пълната поддръжка на Windows и Qt, докато GCC/g++ е логичният избор за Linux и вградени ARM системи като BeagleBone Black. Чрез CMake и прецизно конфигурирани скриптове, проектът може да се изгражда с еднакви изходни кодове и да бъде преносим между архитектури и платформи.

Този подход осигурява висока степен на модулност, независимост от конкретна ОС и позволява разработката да бъде ефективна и мащабируема.

4.3. Реализация на Клиент

Реализацията на клиента в настоящия проект е изградена като вградена C++ програма, изпълнявана върху BeagleBone Black с операционна система Debian Linux^[6]. Основната цел на клиента е да измерва стойности на температура и замърсяване на въздуха (AQI), да реагира на прагови стойности, да изпраща данни до сървъра през TCP/IP и да поддържа надеждно локално логване.

Основна структура и архитектура:

Клиентът е реализиран чрез обектно-ориентиран подход, базиран на три основни модула:

- *ServerConnection* – управлява TCP връзката със сървъра;
- *DataManager* – обединява логика за анализ на данни и поддържа интервали/прагове;
- *FileManager* – осъществява локално логване във файл при нужда.

Допълнително са реализирани помощни класове като TimeProvider, AnalogReader, MutexGuard, и специализирани изключения (AnalogSensorException, FileOpeningException)

- Установяване на TCP връзка (ServerConnection)

Установяването на TCP връзка е критичен етап в работата на клиента, тъй като осигурява комуникационния канал между вграденото устройство (BeagleBone Black) и сървърната страна на системата. Тази логика се реализира в класа ServerConnection, който капсулира всички функции, свързани с мрежовата комуникация.

Процесът започва още в конструктора на класа, където се приема IP адресът на сървъра и се извършва първоначална инициализация на член-променливите, включително флагът m_toTerminate, използван по-късно за безопасно спиране на комуникацията. Веднага след това се извиква методът connectToServer(), който стартира реалната инициализация на TCP сокета.

```
ServerConnection::ServerConnection(const std::string& serverIp)
    : m_serverIp(serverIp), m_toTerminate(false),
    m_dataManager(nullptr) {
    connectToServer();
}
```

В connectToServer() се използва ниско ниво POSIX API за създаване на сокет чрез извикване на socket(AF_INET, SOCK_STREAM, 0). Създаденият сокет е от тип TCP (SOCK_STREAM), използва IPv4 (AF_INET), и по подразбиране работи с протокол TCP. След това се попълва структурата sockaddr_in, в която се задават IP адресът на сървъра (конвертиран чрез inet_pton), портът за комуникация (напр. 54000), и семейството от протоколи.

```

void ServerConnection::connectToServer() {
    m_clientSocket = socket(AF_INET, SOCK_STREAM, 0);
    // sockaddr_in setup ...
    if (connect(m_clientSocket, (sockaddr*)&m_serverAddr,
sizeof(m_serverAddr)) < 0)
        throw ConnectionException("Cannot connect to server");
}

```

Ако connect() не успее, се генерира изключение от тип ConnectionException, което прекратява стартирането на клиента. Това осигурява ясно отделяне на комуникационния слой от логиката за управление на данни и улеснява обработката на грешки.

След успешно свързване, клиентът незабавно преминава към процедура по регистрация. Това включва изпращане на команден идентификатор към сървъра (напр. 0x01 за регистрация) и изчакване на потвърждение. Регистрацията се извършва чрез метода registerClient(), който изпраща уникалния идентификатор на клиента или сигнал за начална инициализация, и checkRegistration(), който обработва отговора от сървъра и преценява дали е успешно регистриран.

По този начин ServerConnection не само създава и управлява мрежовата връзка, но също така задава началните условия за последваща комуникация и контрол, което гарантира сигурен и предсказуем старт на клиента.

- Измерване и събиране на данни (DataManager)

След като клиентът успешно се регистрира в сървъра, управлението се предава на класа DataManager, който играе ключова роля в процеса на събиране, обработка и предаване на измервателни данни. Основната функция на този компонент е да извършва периодични измервания чрез сензори, да оценява резултатите спрямо зададени прагове и при необходимост – да предприема действия, включително изпращане на данни към сървъра и локално записване във файл.

Работният цикъл на DataManager се основава на метода generateData(), който създава инстанция на структурата ClientData, съдържаща GPS координати, температура и стойност за качеството на въздуха (AQI). В проекта координатите са фиксирани (например latitude = 42.6977, longitude = 23.3219), докато температурата и AQI се извличат в реално време чрез AnalogReader.

Класът AnalogReader работи директно със специфичните аналогови входове на BeagleBone Black (напр. AIN0 и AIN1), откъдето чете аналоговите стойности, идващи от свързаните сензори. Температурният сензор MCP970X^[9] е свързан към един от тези входове и предоставя аналогово напрежение, което се конвертира във валидна температура чрез функцията voltageToNumber(). Аналогично, газовият сензор MQ-135^[8] осигурява напрежение, което се интерпретира като индекс на качеството на въздуха

(AQI). При всяко измерване стойностите се оразмеряват и закръглят така, че да бъдат представими чрез short типове в структурата ClientData.

След извличане на данните, се извършва проверка дали AQI надвишава зададения праг чрез метода isNotificationNeeded(short aqi). Ако стойността е в критичен диапазон, DataManager предприема две действия: първо, изпраща измерването към сървъра чрез sendToServer(data), и второ – го записва във файл чрез FileManager:

```
if (isNotificationNeeded(generatedData.aqi)) {  
    sendToServer(generatedData);  
    m_fileManager.writeData(generatedData);  
}
```

Методът sendToServer() използва наличната инстанция на ServerConnection, за да форматира структурата ClientData в бинарна форма и да я изпрати по вече установената TCP връзка. Пакетът може да бъде съпроводен от код за операция, идентифициращ го като нотификация.

В същото време, FileManager записва данните в локален текстов файл, който служи като резервен лог. Това е особено важно при нестабилна мрежа или за целите на последващ анализ. Достъпът до файловата система е синхронизиран чрез std::mutex, за да се предотвратят конфликти при едновременен достъп от различни потоци.

Целият този процес е обгърнат от непрекъснат работен цикъл, който се управлява чрез флаг m_toTerminate, позволяващ безопасно спиране при прекъсване на връзката или при команда от сървъра.

С този модул клиентът реализира пълния цикъл на измерване – от сензорите, през логическата обработка и решението за нотификация, до комуникацията със сървъра и съхранението на данните.

- Извличане на аналогови стойности (AnalogReader)

Класът AnalogReader играе критична роля в хардуерната интеграция на клиента, като осъществява директен достъп до аналоговите пинове на BeagleBone Black за извличане на стойности от свързаните сензори. BeagleBone предоставя интерфейс за достъп до аналоговите входове чрез виртуалната файлова система в Linux, по-точно чрез директориите под /sys/bus/iio/devices/iio:device0/. Всеки аналогов вход (AIN0–AIN6) има съответен файл, например in_voltage1_raw за AIN1. Четенето на стойност от такъв файл връща цели числа (обикновено между 0 и 4095), представляващи стойности от 12-битовия аналогово-цифров преобразувател.

Методът readAIN(int ainIndex) на класа AnalogReader отваря и чете такъв файл, след което връща резултата като int. На тази база се изграждат специализирани методи като readTemperature() и readAQI(), които интерпретират суровите стойности в реални физични величини.

Пример за измерване на температура чрез свързания термистор MCP970X (на AIN1):

```
short AnalogReader::readTemperature() {
    int raw = readAIN(1); // AIN1 = MCP970X
    float voltage = raw * (1.8 / 4096);
    return static_cast<short>((voltage - 0.5) * 100); // Температура
    в °C
}
```

Тук суровата стойност raw се умножава по $(1.8 / 4096)$, за да се получи напрежението в диапазона 0–1.8 V, което е номиналният входен диапазон на BeagleBone Black. Формулата $(voltage - 0.5) * 100$ преобразува напрежението в температура в градуси по Целзий, като се използва характеристиката на MCP970X^[9], при която 500 mV (0.5 V) съответстват на 0 °C, а всяко следващо повишение с 10 mV съответства на 1 °C.

По аналогичен начин се реализира и четенето на стойности за AQI, като се използва друг аналогов вход – AIN0, свързан към газов сензор MQ-135^[8]. Тъй като MQ-135 не връща директно AQI, а напрежение, то се мащабира и преобразува в диапазон от 0 до 500, съответстващ на стандартната скала на индекс за качество на въздуха. Например:

```
short AnalogReader::readAQI() {
    int raw = readAIN(0); // AIN0 = MQ-135
    float voltage = raw * (1.8 / 4096);
    return static_cast<short>(voltage * 278); // Скалиране към 0-500
}
```

Факторът за скалиране (в случая приблизително 278) е получен експериментално, така че максималният възможен вход (около 1.8 V) да се преобразува до стойност близка до 500 AQI.

В допълнение, AnalogReader включва базова обработка на изключения. Ако файлът не може да бъде отворен (например ако няма достъп до AIN директорията), се хвърля потребителско изключение от тип AnalogSensorException, което позволява на DataManager да предприеме подходящи действия – например логване на грешка и прекратяване на измерванията.

Цялостната реализация на AnalogReader гарантира надеждна и прецизна комуникация със сензорния хардуер на BeagleBone, предоставяйки основните измервателни данни за останалата част от клиента.

- Локално логване на данни (FileManager)

Класът FileManager е отговорен за локалното съхранение на измерванията, които клиентът генерира и изпраща към сървъра. Това е важен елемент от устойчивостта на

системата – дори при временна загуба на връзка или грешки при изпращане, данните не се губят, а се архивират във файл, който може да бъде анализиран по-късно или препратен при повторно свързване.

Основният метод за запис е `writeData(const ClientData& data)`, който получава структура със стойности за температура, AQI, както и фиксирани координати. За да се гарантира безопасен достъп от няколко нишки едновременно, методът използва `std::lock_guard<std::mutex>` върху `m_fileMutex`, така че само една нишка да има достъп до файла в даден момент. Това е особено важно, тъй като измерванията могат да се извършват паралелно с други операции, като например спиране на връзката или изпращане на команди от сървъра.

Вътрешният поток `m_output`, който представлява обект от тип `std::ofstream`, се използва за директен запис на данните във файл. Данните се формират в CSV стил – всяко измерване представлява един ред с полета, разделени със запетаи. Форматът на записа включва времева отметка, извлечена чрез вградения обект `TimeProvider`, който връща текущия час във формат `hh:mm:ss`. Примерен ред във файла може да изглежда така:

```
14:23:51, 42.6977, 23.3219, 27, 134
```

Това съхранява: час на измерването, географска ширина, географска дължина, температура в градуси по Целзий и индекс на качество на въздуха.

Допълнителна функционалност на класа е методът `sendTerminationSignal()`, който се извиква в края на сесията – например когато сървърът подаде команда за изключване или при детекция на загубена връзка. Този метод записва специален ред във файла (например `“== END OF SESSION ==”`), който ясно отделя сегментите от данни и маркира завършването на текущата комуникационна сесия.

Конструкторът на `FileManager` създава или отваря изходен файл с уникално име, базирано на идентификатора на клиента (например `"client_123.txt"`), което позволява по-късно тези файлове да бъдат асоциирани с конкретни записи в базата данни на сървъра. Имената на файловете могат да бъдат подадени чрез `ServerConnection` при инициализация.

Целият подход на `FileManager` осигурява локално резервиране на всички важни данни, спазвайки принципите за синхронизация, устойчивост при грешки и ясно логическо структуриране на измерванията.

- Работа с интервали и прагове

Комуникацията между сървър и клиента не се ограничава само до изпращане на измервания – тя включва и двупосочен контрол, при който сървърът може да задава параметри, влияещи на поведението на клиента в реално време. Два от основните параметъра, които могат да бъдат променени по този начин, са *интервалът между измерванията* (наричен `sampleTime`) и *прагът на замърсяване* (наричен `threshold`), който определя кога едно измерване следва да бъде изпратено незабавно към сървъра.

Когато сървърът желае да промени интервала на измерване, той изпраща команда към клиента, съдържаща новото време в милисекунди като стойност от тип `short`. На

страната на клиента, това се обработва от метода `recvSampleTime()` в класа `ServerConnection`. Там новата стойност се приема през TCP сокета с помощта на стандартната POSIX функция `recv()`, която прочита точно 2 байта (размера на `short`) от входящия поток:

```
void ServerConnection::recvSampleTime() {
    short newTime;
    recv(m_clientSocket, (char*)&newTime, sizeof(newTime), 0);
    m_dataManager.setSamplePeriod(newTime);
}
```

След получаването ѝ, новият интервал се подава към инстанцията на `DataManager`, чрез извикване на метода `setSamplePeriod()`. Този метод актуализира вътрешната стойност `m_samplePeriod`, която се използва от основния работен цикъл в `DataManager`, за да определи колко време да изчака между две последователни измервания. Така новият интервал започва да се прилага веднага, без нужда от рестартиране или повторно свързване.

Сходен е и механизмът за промяна на прага за замърсяване:

```
void ServerConnection::recvThreshold() {
    short newThreshold;
    recv(m_clientSocket, (char*)&newThreshold, sizeof(newThreshold), 0);
    m_dataManager.setThreshold(newThreshold);
}
```

Тук отново се приема 2-байтова стойност чрез `recv()`, която представя новия праг (например `AQI >= 150`). Тази стойност се записва в `DataManager` чрез `setThreshold()`, който обновява променливата `m_threshold`. Тази стойност се използва в метода `isNotificationNeeded(short aqi)`, който проверява дали дадено измерване трябва да бъде сметнато за значимо и изпратено незабавно към сървъра.

По този начин клиентът поддържа динамична конфигурируемост, позволявайки на сървъра да контролира честотата на измервания и чувствителността към замърсяване спрямо конкретни нужди или ситуации (например аварийно замърсяване, промяна в мрежовия трафик или оптимизация на ресурси). Това увеличава адаптивността на системата и позволява централизирано управление на множество устройства в реална експлоатационна среда.

- Основен работен цикъл

Основният работен цикъл на клиента се реализира в метода `handleConnection()` на класа `ServerConnection`. Това е централната логика, която управлява цялата комуникация със сървъра след успешното установяване на TCP връзка. Цикълът работи непрекъснато, докато флагът `m_toTerminate` не бъде активиран – например при прекъсване на връзката или по команда за изключване. Основната му функция е да следи за входящи команди от сървъра, да ги разпознава по уникален идентификатор (символ) и да задейства съответното действие.

Всяка команда от сървъра се приема чрез POSIX функцията `recv()`, която очаква точно 1 байт — това е символът, идентифициращ операцията. Когато бъде приета команда '0', това означава, че сървърът иска да промени интервала между измерванията. Извиква се `recvSampleTime()`, който очаква следващите 2 байта от сокета (тип `short`) и ги подава към `DataManager` чрез `setSamplePeriod()`.

Аналогично, команда '1' означава промяна на прага за известяване. Извиква се `recvThreshold()`, който също очаква 2 байта и ги подава към `setThreshold()` в `DataManager`.

Команда '2' сигнализира на клиента да изчисти натрупаните временни измервания. Това може да се използва например при рестарт или заявка за нова серия от данни. Методът `clearData()` в `DataManager` се грижи да изтрие вътрешни буфери или временни стойности.

Команда '3' задейства изпращане на локално записания лог файл обратно към сървъра. Това се извършва чрез `sendFile()` в `FileManager`, който отваря файла, чете ред по ред и го предава през сокета, добавяйки сигнал за край на прехвърлянето.

Тази модулна структура на командите позволява лесно разширяване — добавянето на нова команда се свежда до дефиниране на нов идентификатор, допълнителен `case` в `switch` блока, и съответен метод, който изпълнява операцията. Това прави комуникационния протокол лесен за поддръжка и адаптация, като същевременно гарантира бърза реакция на клиента спрямо управленски действия от страна на сървъра.

- Сигурност, синхронизация и изключения

В системата на клиента са внедрени няколко ключови механизма за гарантиране на сигурност, синхронизация и коректно управление на грешки, особено в контекста на многопотокови операции и достъп до хардуерни или файлови ресурси. Това е от критично значение, тъй като клиентът работи в реално време, извършва периодични измервания, достъпва физически сензори и обменя данни със сървър по TCP/IP.

За синхронизация между нишки се използва специален помощен клас `MutexGuard`. Той следва RAII парадигмата (`Resource Acquisition Is Initialization`), като автоматично заключва подадения `std::mutex` в конструктора си и го освобождава в деструктора. Това елиминира риска от забравен `unlock()`, който би довел до `deadlock` или непредсказуемо поведение. Например при запис във файл:

```
void FileManager::writeData(const ClientData& data) {
    std::lock_guard<std::mutex> lock(m_fileMutex);
    m_output << ...;
}
```

В този фрагмент `std::lock_guard` действа по същата логика като `MutexGuard`, гарантирайки, че достъпът до файлът е нишково безопасен.

За работа с външни устройства, каквито са температурният термистор MCP970X^[9] и газовият сензор MQ-135^[8], клиентът използва директен достъп до файловата система на BeagleBone Black (`/sys/bus/iio/devices/`). Ако възникне грешка при отваряне или четене на тези файлове, се хвърля изключение от тип `AnalogSensorException`. Това позволява на системата да прихване и адекватно да реагира на хардуерни неизправности, вместо да крашне или да запише некоректни стойности.

Подобна логика се прилага и при отваряне на текстови файлове за локален лог — ако файлът не може да бъде отворен или записан, се хвърля `FileOpeningException`. Това дава възможност на повикващия код (например `FileManager` или `DataManager`) да уведоми потребителя или да предприеме алтернативни действия (като създаване на резервно копие или смяна на директорията за запис).

Контролът върху работния поток се реализира чрез булев флаг `m_toTerminate`. Той се проверява периодично в основните цикли (например в `handleConnection()` или `DataManager::run()`) и указва кога нишката трябва да прекрати работа. Това осигурява плавно и безопасно спиране на клиента — спиране на връзката със сървъра, запис на финални данни във файла и освобождаване на ресурси.

Комбинацията от RAII синхронизация, изключения при хардуерни и I/O операции и безопасно управление на нишков живот прави клиента устойчив на грешки, надежден при дългосрочна работа и лесен за поддръжка при евентуално разширяване с нови функции или устройства.

- Формат на данните

Форматът на данните, които клиентът изпраща към сървъра, е строго структуриран чрез `struct ClientData`. Това е компактна C++ структура, която съдържа всички необходими измервателни параметри, свързани с конкретна сесия на клиента. Включва следните полета:

```
struct ClientData {
    double latitude;
    double longitude;
    short aqi;
    short temperature;
};
```

Полетата `latitude` и `longitude` са с тип `double` и съответно съхраняват фиксираните GPS координати на клиента. Те не се променят в реално време и служат за географска идентификация на източника на измерванията. Полетата `aqi` и `temperature` са от тип `short` и отразяват последните отчетени стойности за индекс на замърсяване на въздуха (Air Quality Index) и температура в градуси по Целзий. Те се генерират в реално време чрез хардуерни сензори – MQ-135 за AQI и MCP970X за температурата.

Преди изпращане по TCP, тази структура се сериализира в двоичен формат, който е директно съвместим с байтовата подредба на системата. За да се гарантира, че няма падинг между елементите (който компилаторът може да добави с цел подравняване), се използва `#pragma pack(push, 1)`, което инструктира компилатора да опакова структурата без излишни празни байтове. По този начин цялата структура е точно 20 байта ($8 + 8 + 2 + 2$), и се изпраща чрез:

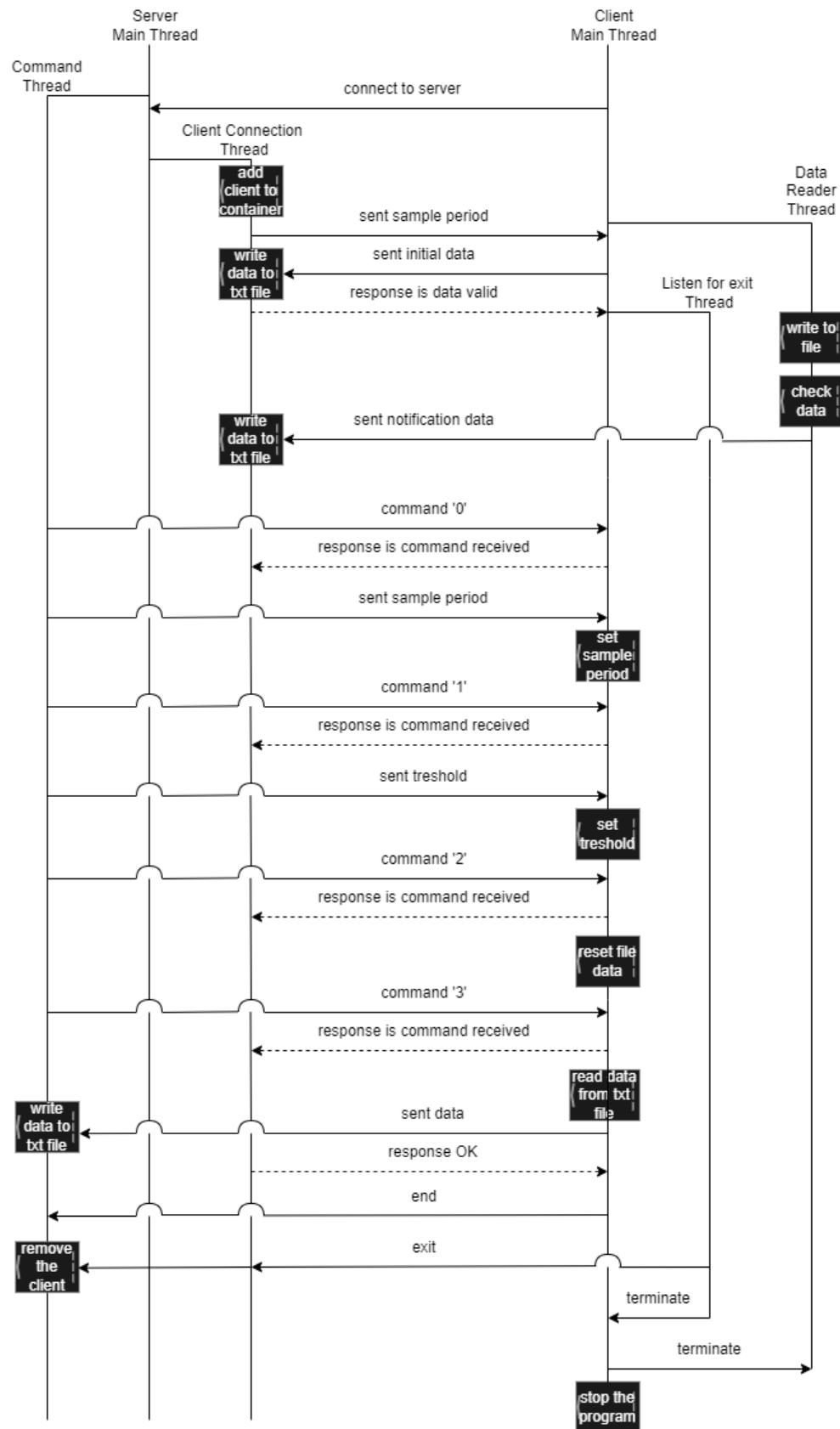
```
send(m_socket, reinterpret_cast<const char*>(&data),  
sizeof(ClientData), 0);
```

Тъй като структурата е в бинарен формат, тя не разчита на текстови протоколи или парсинг, което я прави изключително ефективна при предаване на данни с ниска латентност и минимален overhead. Все пак, тази оптимизация предполага, че клиентът и сървърът трябва да работят на системи със съвместими архитектури (например little-endian), или трябва да се осигури допълнителна обработка (напр. `htons()` / `ntohs()`) при нужда от преносимост.

Предимството на тази схема е високата производителност при предаване на големи обеми от данни в реално време, като същевременно структурата остава лесна за логване, визуализация и запис както в локални файлове, така и в бази данни.

4.4. Реализация на сървър

Диаграмата показва комуникацията между клиент и сървър



Фигура 4.1 – Диаграма на последователностите

Разработката на сървърната част на проекта е реализирана чрез C++ и Qt, като обединява контролна логика, графичен интерфейс и връзка с MS SQL база данни. Тук ще представим подробно всеки основен компонент – контролера (мрежова комуникация и обработка на клиенти), базата данни (чрез DatabaseManager) и графичния интерфейс (реализиран с Qt и интегриран с контролера чрез сигнали и слотове).

- Контролерна логика: Server и ClientConnection

Контролерната логика на сървъра е изградена върху многопоточна архитектура, която осигурява стабилна и ефективна комуникация с множество клиенти, без да блокира графичния потребителски интерфейс. Главният сървърен обект (Server) се създава от графичния интерфейс (MainWindow) и се премества в отделен QThread, така че неговата работа по приемане на нови клиенти и обработка на данни да протича фоново и независимо от визуалната част на приложението. Когато нишката стартира, се извиква методът start(), който създава TCP сокет, свързва се към зададен порт и активира метода acceptConnections() в отделна C++ нишка, където сървърът започва да слуша за входящи клиентски връзки.

Всеки нов клиент се приема чрез accept и създаденото клиентско гнездо се обвива в обект от тип ClientConnection. Този обект се стартира в нова нишка и започва да обслужва конкретния клиент – получава команди, измервателни данни, и изпраща отговори. Класът ClientConnection съдържа метод receiveData(), който приема данни във формат ClientData (съдържащ координати, температура и AQI), валидира размера и при успешно приемане – извиква saveMeasurement(). Отговорът към клиента се изпраща под формата на еднобайтов код за потвърждение, който информира клиента за успешно обработване на измерването.

стартиране на клиентска нишка:

```
m_acceptThread = std::thread([this] { acceptConnections(); });
```

Паралелно с това, получените измервания се записват едновременно в текстов файл и в база данни. Записът във файл се извършва със защита чрез std::mutex, така че писането да бъде безопасно при многопоточна работа. За връзката с базата се използва DatabaseManager, който е реализиран чрез ODBC^[2] интерфейс. При инициализацията се създават и конфигурират съответните дескриптори за среда, връзка и запитвания. Системата се свързва към база данни WeatherStationDB, хоствана в локален SQL Server, използвайки драйвера {SQL Server}.

При всяко ново измерване, DatabaseManager изпълнява SQL INSERT заявка към таблицата NOTIFICATED_RECORDINGS, като се записват координатите, температурата, AQI стойността и идентификаторът на клиента. Този клиентски идентификатор се съпоставя с таблицата CLIENTS, където се съдържа и името на съответния текстов файл за логване. Ако клиентът не е регистриран предварително, системата автоматично го добавя в таблицата чрез INSERT INTO CLIENTS.

В случаите, когато потребителят от графичния интерфейс избере „Покажи данни“ за определен клиент, DatabaseManager извлича всички записани измервания за този клиент от таблицата NOTIFICATED_RECORDINGS. Данните се връщат във вид, удобен за показване на екрана или визуализация чрез графики. При нужда се поддържа и възможност за ръчни заявки, които се записват в таблицата REQUESTED_RECORDINGS.

Тази архитектура осигурява надеждност, добра разделимост на логиката (контролер – база – интерфейс) и възможност за мащабируемост. Всеки компонент работи в своята отговорност, като комуникацията между тях се осъществява чрез сигнали, събития и ясно дефинирани интерфейси.

- Интеграция с база данни: DatabaseManager

Интеграцията с базата данни в проекта се осъществява чрез класа DatabaseManager, който използва стандартния ODBC^[2] интерфейс, предоставян от операционната система. Това решение осигурява гъвкавост и независимост от конкретен доставчик на база данни, като в случая се използва Microsoft SQL Server. При инициализация на връзката се създава ODBC^[2] среда с помощта на SQLAllocHandle за SQL_HANDLE_ENV и SQL_HANDLE_DBC. След това чрез SQLSetEnvAttr се задава версията на ODBC^[2] (в случая ODBC 3.0^[2]), а SQLDriverConnect се използва за установяване на връзката към конкретната база данни WeatherStationDB:

```
SQLDriverConnect(..., "DRIVER={SQL  
Server};SERVER=localhost;DATABASE=WeatherStationDB;");
```

При получаване на измерване от клиент, DatabaseManager извършва запис в таблицата NOTIFICATED_RECORDINGS. Методът insertNotification изгражда SQL заявка чрез std::stringstream, съдържаща координатите, температурата, AQI и идентификатора на клиента. След това се извиква помощен метод execQuery, който изпълнява заявката:

```
query << "INSERT INTO NOTIFICATED_RECORDINGS (lat, lon, temp, AQI,  
ClientID) VALUES (...);"  
execQuery(query.str());
```

За всеки нов клиент се поддържа проверка чрез clientExists(), която определя дали вече има съществуващ запис в таблицата CLIENTS. Ако няма, се създава нов запис със съответния идентификатор и свързаният текстов файл за логване на измерванията:

```
std::string insert = "INSERT INTO CLIENTS (ClientID, NameOfTextFile)  
VALUES (...);"  
execQuery(insert);
```

Когато потребителят от графичния интерфейс избере бутон „Покажи данни“ за даден клиент, се използва методът `fetchClientData(clientId)`, който извлича всички записи от `NOTIFICATED_RECORDINGS`, свързани с конкретния клиент. Данните се обработват и форматират като списък от стрингове, които се използват за визуализация или анализ в интерфейса. Това осигурява лесен и ефективен достъп до историческите данни на всеки клиент, като същевременно структурата на базата остава ясна и добре организирана.

Цялостната реализация на връзката с базата данни чрез ODBC^[2] е добре адаптирана за използване в Windows среда и предлага стабилна и разширяема архитектура за съхранение и извличане на информация, генерирана от реални хардуерни сензори.

- Графичен интерфейс (GUI)

Графичният потребителски интерфейс (GUI) на сървъра е реализиран с помощта на *Qt* 6, като централната част на визуализацията е класът `MainWindow`. Интерфейсът е проектиран чрез *Qt Designer*, като след това `.ui` файлът автоматично се компилира чрез `CMake` благодарение на `AUTOUI`, което позволява лесна интеграция без нужда от ръчно свързване на елементи.

Целта на GUI-то е да осигури интуитивен контрол над сървъра и клиентските връзки, както и да предлага преглед на реално време на измерванията на температура и AQI. Основните компоненти на интерфейса включват:

- *clientListWidget*, който показва списък с в момента свързани клиенти. Всеки нов клиент се добавя динамично чрез слотове, които се задействат при сигнал от `Server` класа.
- *sampleTimeEdit* и *thresholdEdit*, които позволяват въвеждане на интервал на измерване и праг за известие. След потвърждение, въведените стойности се изпращат към избрания клиент чрез сървъра.
- *temperatureChartView* и *aqiChartView*, които визуализират стойностите на температура и AQI с помощта на `QChartView`, `QChart` и `QLineSeries`.

За поддържане на диаграмите актуални, се използва `QTimer`, който извиква метода `updateCharts()` на всяка секунда:

```
m_chartUpdateTimer->start(1000);
```

Методът `updateCharts()` извиква функция от `DatabaseManager`, която връща последните 50 измервания за текущо избрания клиент. Върнатите стойности се добавят към съответните графики:

```
std::vector<std::pair<float, int>> data =  
g_dbManager.fetchLastNotificationData(clientId, 50);
```

Взаимодействието между сървъра и интерфейса е реализирано чрез *Qt* *сигнално-слотов механизъм*, което осигурява реактивност и отговаря при свързване или прекъсване на клиент:

```
connect(m_server, &Server::clientConnected, this,
&MainWindow::onClientConnected);
```

Когато потребителят избере команда като „Задай интервал“ или „Изтегли лог“, интерфейсът използва текущо избрания клиент от списъка и извиква съответен метод от *Server*, който изпраща подходяща команда по TCP до клиента.

Допълнителна функционалност включва бутони като:

- „Изтрий лог“ – изпраща команда към клиента за изчистване на логовете;
- „Изтегли лог“ – инициира заявка за изпращане на текстовия лог файл;
- „Покажи данни“ – използва *DatabaseManager*, за да извлече и визуализира всички записи от базата.

От системна гледна точка, проектът е напълно автоматизиран чрез *build.bat* скрипт, който изтрива стария *build*, създава нова директория, стартира *CMake* и извършва компилацията. Този подход улеснява разработката и разгръщането на проекта в различни среди.

CMakeLists.txt на проекта е конфигуриран да включва всички *Qt* специфики чрез *AUTOMOC*, *AUTOUI* и *AUTORCC*. Изходният код е модулно структуриран в три основни директории:

- *controller* – съдържа логиката на сървъра и клиентите;
- *gui* – графичен интерфейс и всички свързани компоненти;
- *database* – логика за взаимодействие с базата чрез *ODBC*^[2].

Тази ясно разделена архитектура допринася за поддръжка, разширяемост и независимост между отделните функционалности. GUI интерфейсът изпълнява успешно ролята на свързващо звено между потребителя, сървъра и хардуера, предоставяйки пълноценен контрол и мониторинг в реално време.

4.5. Реализация на хардуерна система

- Централен контролер: BeagleBone Black (BBB)

BeagleBone Black (BBB) служи като централен изчислителен и контролен модул в реализираната измервателна система. Това е едноплатков компютър, базиран на 1GHz ARM Cortex-A8 процесор и съдържащ 512MB DDR3 RAM, 4GB eMMC флаш памет, HDMI изход, Ethernet порт и богат набор от GPIO, PWM, I2C, SPI и най-важното за проекта – аналогови входове (AIN).

Операционната система, използвана на BBB, е *Debian Linux*^[6], която е предварително инсталирана в eMMC паметта или заредена от microSD карта. Debian^[6] осигурява пълна Linux среда с поддръжка за разработка на C/C++ код, включително GCC (MinGW за Linux), GDB за дебъгване, POSIX API и възможност за компилация чрез CMake. Това позволява стартиране на клиентското приложение директно на устройството, без нужда от външни конвертори или драйвери.

Една от ключовите функционалности на BBB е наличието на *вградени аналогови входове (AIN)*, което го отличава от повечето Raspberry Pi-подобни платформи. Аналоговите пинове са свързани с 12-битовия ADC модул на чипа TI Sitara AM335x. Това означава, че аналоговите напрежения в обхват 0 – 1.8V могат да бъдат дигитализирани със стойности между 0 и 4095.

Linux осигурява достъп до ADC чрез *файловата система*, по-точно в директорията:

```
/sys/bus/iio/devices/iio:device0/
```

Вътре се съдържат файлове от типа:

- `in_voltage0_raw` – чете стойността на AIN0 (използван за MQ-135),
- `in_voltage1_raw` – чете стойността на AIN1 (използван за MCP970X).

Четенето на стойност става с отваряне на файл (`std::ifstream`) и парсане на прочетения текст до число. Например, `AnalogReader` реализира функция `readAIN(int channel)`, която съставя името на съответния файл и извлича неговата стойност като цяло число. Това число се мащабира според референтното напрежение (1.8V), за да се получи реална физическа стойност (температура или AQI).

Пример за път към файл:

```
std::string path =  
"/sys/bus/iio/devices/iio:device0/in_voltage1_raw";
```

Тази архитектура е изключително подходяща за *embedded* приложения, където надеждното четене на аналогови сензори и ниската латентност са от ключово значение. В допълнение, наличието на Ethernet порт и възможност за TCP/IP комуникация позволява директно свързване към сървър, без необходимост от допълнителни модули като Wi-Fi или USB към Ethernet адаптери.

BBB е програмиран изцяло на C++, като се използва POSIX съвместим `socket API`^[13] за TCP комуникация, `std::thread` за паралелно извличане на данни, както и файлов

I/O за логване. Това осигурява пълна интеграция между хардуерните сензори и мрежовата комуникация в реално време.

- MCP970X — Температурен сензор

Сензорът *MCP9700/MCP9701* е аналогов температурен сензор, който издава напрежение пропорционално на температурата. Изходното напрежение се подава към *AIN1* (аналогов вход 1) на BBB.

- *Захранване*: 3.3V, подавано от BBB.
- *Изход*: аналогово напрежение, вариращо от ~0.5V при 0°C до ~1.0V при 50°C.
- *Формула за преобразуване*:

```
short AnalogReader::readTemperature() {  
    int raw = readAIN(1); // AIN1  
    float voltage = raw * (1.8 / 4096); // ADC е 12-bit, Vref = 1.8V  
    return static_cast<short>((voltage - 0.5) * 100); // резултат в  
    °C  
}
```

Тук $1.8 / 4096$ представлява скала за преобразуване на ADC стойности в реални напрежения. MCP970X дава $10\text{ mV}/^{\circ}\text{C}$ чувствителност.

MQ-135 — Сензор за качество на въздуха

MQ-135 е сензор, използван за откриване на вредни газове (амоняк, бензен, CO₂ и други). Това е аналогов сензор с изход, зависим от концентрацията на замърсители във въздуха.

- *Захранване*: 5V (отделен регулатор или захранващ пин на BBB).
- *Изход*: аналогово напрежение (0–5V), в зависимост от замърсяването.
- *Връзка към BBB*: чрез *AIN0*, но чрез делител на напрежение, понеже ADC на BBB приема максимум 1.8V.

Делител на напрежение:

MQ-135 изходът се намалява чрез резистивен делител:

```

Vout (към BBB AIN0)
|
R2 (10kΩ)
|
+-----> GND
|
R1 (20kΩ)
|
MQ-135 Out
|
+5V

```

Това ограничава напрежението до стойности в рамките на 0–1.67V, безопасни за AIN0.

Прочит на AQI:

```

short AnalogReader::readAQI() {
    int raw = readAIN(0); // AIN0
    return static_cast<short>((raw / 4096.0) * 500); //
резултат в условна AQI скала (1-500)
}

```

Тук се използва **линейно мащабиране** до AQI диапазон. В практиката, реалният отговор на MQ-135 е нелинеен, но тук се използва опростен модел за нуждите на проекта.

- Електрозахранване
 - MCP970X се захранва директно от *BBB 3.3V пин*.
 - MQ-135 изисква 5V, която се подава от BBB или външен източник.
 - Общата консумация на сензорите е малка (~200 mW), подходяща за непрекъснатата работа.
- Връзки и пинове

Таблица 5: **Gas Sensor MQ-135 връзка с Beaglebone Black**

MQ-135 пин	BBB пин
VCC	P9_5 (5V)
GND	P9_1 or P9_2 (GND)
AOUT	P9_39 (AIN0)

Таблица 6: **Termistor MCP970X връзка с Beaglebone Black**

MCP970X пин	BBB пин
VCC	P9_3 (3.3V)
GND ADC	P9_34 (GND ADC)
AOUT	P9_40 (AIN1)

- *Работа на хардуера*

При стартиране на клиента, класът AnalogReader периодично чете аналоговите стойности от `/sys/bus/iio/devices/iio:device0/in_voltageX_raw`, преобразува ги и ги изпраща към DataManager. DataManager анализира стойностите, проверява дали AQI надвишава праг и при нужда, ги праща на сървъра.

Цялата система разчита на точна конфигурация на аналоговите входове, правилно мащабиране на напреженията и стабилно захранване, за да гарантира коректни и безопасни измервания. Хардуерът е проектиран така, че да работи непрекъснато, като изпраща стойности по зададен интервал и реагира на команди от сървъра (за промяна на интервал, изчистване на лог и т.н.).

Глава 5. РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ

Какво представлява системата:

- Системата е интелигентна станция за наблюдение на въздуха и температурата, състояща се от две основни части: *устройство за измерване и графичен интерфейс на сървъра*. Измервателното устройство (поставено на терен) събира данни чрез сензори, изпраща ги автоматично до сървъра, където потребителят има достъп до тях чрез визуален интерфейс.

Стартиране на системата:

- Включване на клиентското устройство

Устройството се стартира автоматично при подаване на захранване. То е свързано чрез кабел към мрежа (Ethernet) и автоматично открива и се свързва със сървъра.

- Стартиране на сървъра

На компютъра, който изпълнява сървъра, отворете файла `run.bat` с двоен клик. Ще се стартира *графичният интерфейс*, който управлява работата на системата.

Работа с интерфейса:

- Свързани клиенти

В лявата част на прозореца има списък с всички свързани клиентски устройства. Всеки клиент е отбелязан с уникален идентификатор.

- Настройка на интервал

Полето „Интервал (ms)“ позволява да зададете колко често клиентът да прави измервания (например: 2000 = на всеки 2 секунди, което е интервалът по подразбиране, в случай, че не зададете стойност). Въведете стойността в полето „*Sample Time*“ и натиснете бутона „*Set*“, за да приложите стойността.

- Настройка на праг

Полето „Праг (AQI)“ определя от коя стойност нататък клиентът да праща автоматично измервания. Въведете стойност в полето „*AQI Threshold*“ и натиснете „*Set*“, за да го промените.

- „*Download Log*“ – Сваля файла с всички записани измервания от клиента.
- „*Reset Log*“ – Изтрива натрупаните данни в лог файла на клиента.
- „*Show Data from Client*“ – показва записаните данни до момента, за конкретния клиент, в текстово поле

„*Show Charts*“ зарежда измерванията на избрания клиент. Данните ще се покажат в две графики:

- Графика на температурата (*Temperature Over Time*) – проследява промените в градусите; (Фигура 5.7 и Фигура 5.8)

- Графика на AQI (*AQI Over Time*) – показва колко замърсен е въздухът. (Фигура 5.7 и Фигура 5.8)

Поведение при загуба на връзка:

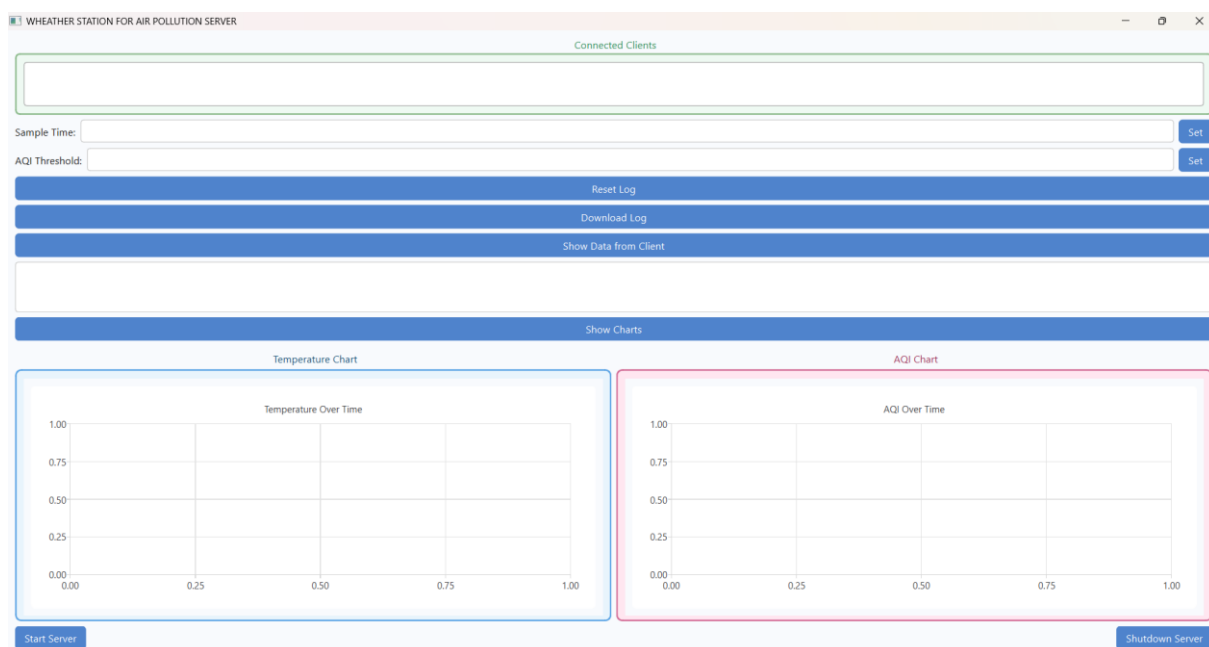
Ако клиентското устройство се изключи или прекъсне връзката:

- Изчезва от списъка с активни клиенти;
- Ресурсите за него, заделени в сървърната част, се зачистват.

Примерен сценарий на употреба:

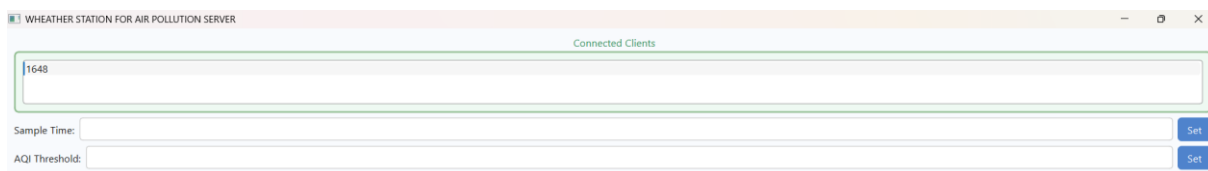
1. Стартирате интерфейса;
2. Устройството се свързва автоматично;
3. Задавате интервал на измерване на 3000 ms и праг AQI = 150;
4. Преглеждате получените стойности в реално време;
5. Изтегляте лог файл за архив;
6. При нужда изключвате системата, всички клиенти автоматично прекратяват работа.

Системата е стартирана и готова за работа:



Фигура 5.1 – Начално състояние на системата

Когато клиент се свърже се показва на екрана със своя идентификационен номер (в случая 1648):

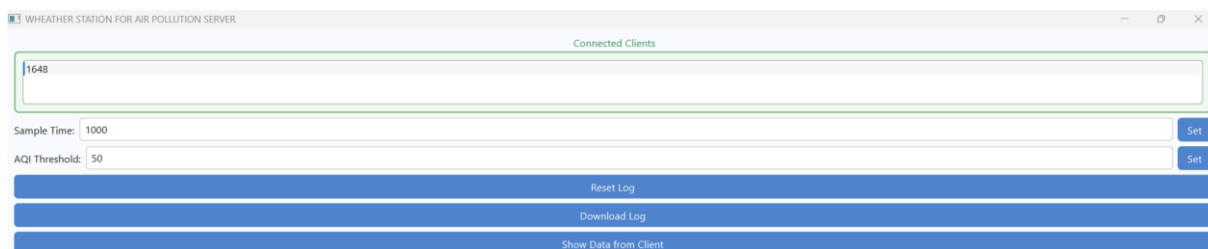


Фигура 5.2 – Визуализация на един свързан клиент

Клиента, с който искаме да комуникираме се маркира. В полетата (Фигура 5.3) са зададени стойности за *интервал*, през който клиента да прави измервания и *трейшхолд* стойност за качество на въздуха, над която да бъдат изпращани нотификации към сървъра.

Нотификацията представлява цялостен запис – *температура*, *индекс* (качество на въздуха), географска *ширина* и *дължина*.

При натискане на „Set“ бутоните заявките се пращат на клиента.



Фигура 5.3 – Задаване на стойност на *интервал* и *трейшхолд* на клиент

Получените данни се записват в текстов файл през цялото време (Фигура 5.4):

```
42.69/23.31/56/26
42.69/23.31/64/26
42.69/23.31/59/26
42.69/23.31/59/26
42.69/23.31/59/26
42.69/23.31/58/26
42.69/23.31/59/26
42.69/23.31/59/26
42.69/23.31/60/26
42.69/23.31/57/26
```

Фигура 5.4 – Част от текстов файл на клиент, локален за сървъра. Нотификации.

Структурата представлява:

Географска ширина/Географска дължина/Индекс/Температура.

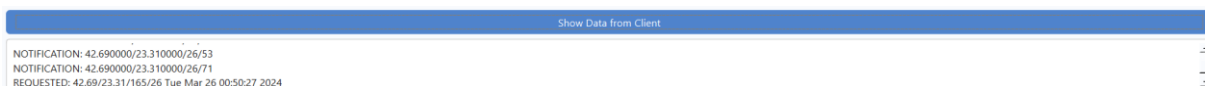
Всеки нов ред е нов запис, иначе казано – ново измерване от клиента, пратено към сървъра.

Натискането на бутона „*Download Log*“ ще поиска всички записани данни от страна на клиента и ще ги запише във съответния текстов файл в сървърната част. Надписът в началото на конкретни лог „*Requested data:*“ (Фигура 5.5) обозначава, че следващите записи не са нотификации, а поискан лог. Поисканите данни се изпращат с ден, час, дата и година към сървъра.

```
378 Requested data:
379 42.69/23.31/166/26 Tue Mar 26 00:50:25 2024
380 42.69/23.31/165/26 Tue Mar 26 00:50:27 2024
381 42.69/23.31/165/26 Tue Mar 26 00:50:29 2024
382 42.69/23.31/61/26 Tue Mar 26 00:50:31 2024
383 42.69/23.31/64/26 Tue Mar 26 00:50:33 2024
384 42.69/23.31/61/26 Tue Mar 26 00:50:35 2024
385 42.69/23.31/61/26 Tue Mar 26 00:50:37 2024
386 42.69/23.31/60/26 Tue Mar 26 00:50:39 2024
387 42.69/23.31/61/26 Tue Mar 26 00:50:41 2024
388 42.69/23.31/59/26 Tue Mar 26 00:50:43 2024
389 42.69/23.31/60/26 Tue Mar 26 00:50:45 2024
390 42.69/23.31/60/26 Tue Mar 26 00:50:47 2024
391 42.69/23.31/59/26 Tue Mar 26 00:50:49 2024
392 42.69/23.31/82/26 Tue Mar 26 00:50:51 2024
393 42.69/23.31/61/26 Tue Mar 26 00:50:53 2024
394 42.69/23.31/73/26 Tue Mar 26 00:50:55 2024
395 42.69/23.31/59/26 Tue Mar 26 00:50:57 2024
396 42.69/23.31/66/26 Tue Mar 26 00:50:59 2024
```

Фигура 5.5 – Част от текстов файл на клиент, локален за сървъра. Поискани данни.

Бутона „*Show Data from Client*“ ще покаже данните, изпратени от клиента до момента, като съответно ще обозначи техният тип – дали са нотификации или поискан лог.



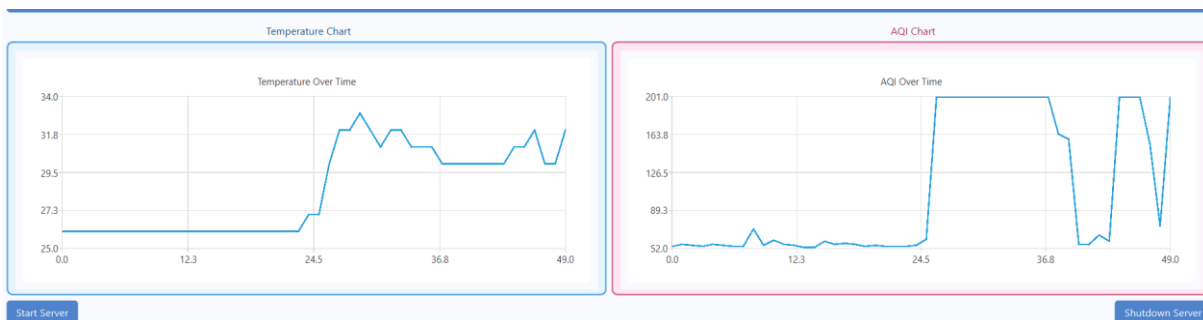
Фигура 5.6 – Визуализирани данни от бутона „Show Data from Client“

След натискане на бутона „*Show Charts*“ се показват графики на изпращаните данни в реално време под формата на нотификации (Фигура 5.7). Диаграмите се базират на последните 50 записа.



Фигура 5.7 – Графики на получените данни в реално време

С цел тестване на системата, сензорите са провокирани да увеличат своите стойности (Фигура 5.8). Температурният сензор е подложен на загряване чрез допир, докато в близост до сензора за качество на въздуха е поставен твърд етилов алкохол, чиито изпарения карат индекса да се повиши.



Фигура 5.8 – Графики на получените данни в реално време. Завишени резултати.

Системата дава възможност за свързване на неограничен брой клиенти наведнъж. На фигурата по долу имаме три клиента, показани на екрана с техните идентификационни номера.

Connect

1648
6828
7072

Sample Time: 1000

AQI Threshold: 50

Reset

Download

Фигура 5.9 – Визуализация на повече от един свързани клиенти

Бутонът “*Reset Log*” (Фигура 5.1) изтрива цялата записана информация в локални за клиента текстов файл (Фигура 5.9).

```

94 212/234/30/23 Mon Jun 16 20:22:55 2025
95 212/234/30/23 Mon Jun 16 20:22:57 2025
96 212/234/30/23 Mon Jun 16 20:22:59 2025
97 212/234/30/23 Mon Jun 16 20:23:01 2025
98 212/234/30/23 Mon Jun 16 20:23:03 2025
99

```

Фигура 5.10 – Част от текстов файл на клиент, локален за клиента. Преди “Reset Log”

Така изглежда файла при клиента след натискането на бутона “*Reset Log*”:

```

2 212/234/30/23 Mon Jun 16 20:23:45 2025
3 212/234/30/23 Mon Jun 16 20:23:47 2025
4 212/234/30/23 Mon Jun 16 20:23:49 2025
5 212/234/30/23 Mon Jun 16 20:23:51 2025
6

```

Фигура 5.11 – Част от текстов файл на клиент, локален за клиента. След “Reset Log”

Старите данни са зачистени и започва записването на нови. Може да се разбере по номера на редовете (Фигура 5.9 и Фигура 5.10).

**За някои опити са използвани тестови клиенти, изпълняващи се на Линукс среда, без хардуер, затова стойностите може да са добавени в код на места, което води до еднакви записи. При свързване на клиент с хардуерна система, получените стойности са реални.*

При натискане на бутона “Shutdown Server” (в долния десен ъгъл - Фигура 5.1) всички клиенти биват изключени прекратяват работа, техните файлове с данни са зачистени от системата. Нов клиент няма възможност да се свърже до натискане на “Start Server” бутона (в долния ляв ъгъл - Фигура 5.1).

AQI (Air Quality Index) – информация

Таблица 7 показва AQI стойностите, според нивото на замърсяване на въздуха и влиянието върху човешкото здраве.

Таблица 7: Класификация на стойностите на *AQI (Air Quality Index)*

AQI стойност	Качество на въздуха	Описание / Влияние върху здравето
0 – 50	Добро	Въздухът е чист и не представлява риск за населението.
51 – 100	Приемливо	Приемливо качество; възможен лек риск за чувствителни хора (астматици, деца, възрастни).
101 – 150	Нездравословно чувствителни за	Чувствителните групи може да изпитат симптоми; останалата част от населението вероятно не е засегната.
151 – 200	Нездравословно	Всеки може да започне да изпитва симптоми; чувствителните групи – по-сериозни.
201 – 300	Много нездравословно	Известна опасност за здравето; сериозни симптоми за чувствителните групи.
301 – 500	Опасно	Спешно здравно предупреждение. Цялото население е засегнато.

**AQI в този контекст е обобщен индекс, но в проекта той се измерва индиректно чрез газовия сензор MQ-135. Стойностите, които той връща, са прецизирани така, че да попадат в този интервал (1–500), след обработка на аналоговия сигнал.*

ЗАКЛЮЧЕНИЕ

Разработената система за мониторинг на атмосферни условия и качество на въздуха представя цялостен и балансиран подход към реализацията на интелигентно решение с реална практическа стойност. Проектът успешно съчетава хардуерни и софтуерни компоненти, реализирани чрез съвременни технологии като C++, Qt, SQL Server, TCP/IP мрежова комуникация и хардуерен интерфейс с BeagleBone Black. Използването на аналогови сензори за температура и качество на въздуха, съвместно с клиентски модули, базирани на Linux, позволява надеждно събиране на данни в реално време. Сървърната част, работеща под Windows, осигурява централизирано управление, визуализация и архивиране на измерванията. Графичният интерфейс улеснява работата на крайния потребител, позволявайки ръчно задаване на параметри, преглед на данни и управление на клиентите без нужда от допълнителна техническа експертиза.

Едно от най-важните качества на системата е нейната надеждност и реактивност. Тя отговаря на промените в околната среда чрез предварително зададени прагове и може автоматично да алармира при наличие на рискови условия. Данните се съхраняват паралелно в локални файлове и в база данни, което гарантира тяхната устойчивост. Интеграцията между клиента и сървъра е реализирана чрез добре дефиниран TCP протокол, с ясно разграничени команди и отговори, което прави системата устойчива и лесна за разширяване.

Освен стабилната архитектура, проектът демонстрира и добра организационна култура – сорс кодът е разделен по функционалност, конфигурационните файлове и CMake системата позволяват лесна компилация и поддръжка, както под Windows, така и под Linux. Това дава възможност системата да бъде лесно внедрена в различни среди – от образователни лаборатории до малки екологични станции.

В перспектива, системата предлага множество възможности за развитие. Тя може да бъде надградена чрез добавяне на нови сензори (влага, налягане, CO₂), интеграция с мобилно приложение или веб интерфейс за дистанционно наблюдение и управление, автоматично генериране на отчети, известия по имейл или SMS, както и използване на облачни технологии за дългосрочно съхранение и анализ на данните. В допълнение, могат да бъдат внедрени алгоритми за прогнозиране на замърсяване или температурни аномалии. Всичко това прави системата не само полезна в текущия ѝ вид, но и отворена към иновации и адаптация към съвременните нужди на умната екологична инфраструктура.

ИЗПОЛЗВАНИ ИЗТОЧНИЦИ

- [1] Microsoft. (n.d.). *C++ language reference*. Microsoft Learn.
[<https://learn.microsoft.com/en-us/cpp/cpp/cpp-language-reference>]
- [2] Microsoft. (n.d.). *ODBC Programmer's Reference*. Microsoft Learn.
[<https://learn.microsoft.com/en-us/sql/odbc/reference/odbc-programmer-s-reference>]
- [3] The Qt Company. (n.d.). *Qt for Developers*.
[<https://doc.qt.io/>]
- [4] ISO/IEC. (2017). *Information technology – Programming languages – C++*. ISO/IEC.
[<https://www.iso.org/standard/68564.html>].
- [5] BeagleBoard.org Foundation. (n.d.). *BeagleBone Black System Reference Manual*.
[<https://beagleboard.org/black>]
- [6] Debian Project. (n.d.). *Debian GNU/Linux Operating System*.
[<https://www.debian.org/>]
- [7] Environmental Protection Agency. (2022). *Air Quality Index (AQI) Basics*.
[<https://www.airnow.gov/aqi/aqi-basics>]
- [8] Bosch Sensortec. (n.d.). *Air Quality Sensors – Technical Documentation*.
[<https://www.bosch-sensortec.com/airquality/>]
- [9] Microchip Technology Inc. (2018). MCP970X. *Thermistor Data Sheet*.
[<https://www.microchip.com/>]
- [10] GitHub. (n.d.). *CMake Documentation*.
[<https://github.com/Kitware/CMake/blob/master/Modules/Documentation.cmake>]
- [11] IEEE. (2020). *Standard for Local and Metropolitan Area Networks*. IEEE 802-2020.
[<https://ieeexplore.ieee.org/document/9018454>]
- [12] Posix.org. (n.d.). *POSIX.1-2017 Standard*.
[<https://pubs.opengroup.org/onlinepubs/9699919799.2018edition/>]
- [13] Socket Programming Tutorials. (n.d.). *Beej's Guide to Network Programming*.
[<https://beej.us/guide/bgnet/>]
- [14] Stack Overflow. (n.d.). *C++ Networking, Multithreading, Qt and SQL discussions*.
[<https://stackoverflow.com/>]
- [15] Microsoft SQL Server.
[<https://www.microsoft.com/en-us/sql-server>]

ПРИЛОЖЕНИЕ

Приложение 1

```
#pragma once
```

```
#include <QObject>
```

```
#include <thread>
```

```
#include <map>
```

```
#include <atomic>
```

```
#include <winsock2.h>
```

```
#include <memory>
```

```
#include "ClientConnection.hpp"
```

```
class Server : public QObject {
```

```
    Q_OBJECT
```

```
public:
```

```
    explicit Server(QObject* parent = nullptr);
```

```
    ~Server();
```

```
    bool start();
```

```
    void shutdown();
```

```
    void sendCommandToClient(int socketId, char commandId, const QByteArray& payload);
```

```
signals:
```

```
    void clientConnected(int socketId);
```

```
    void clientDisconnected(int socketId);
```

```
    void serverStopped();
```

```
private:
```

```

void acceptConnections();
void handleClient(std::shared_ptr<ClientConnection> client);

std::thread m_acceptThread;
std::atomic<bool> m_running;
SOCKET m_serverSocket;
sockaddr_in m_serverAddr;

std::mutex m_clientsMutex;
std::map<int, std::shared_ptr<ClientConnection>> m_clients;
};

```

Приложение 2

```

#include "Server.hpp"
#include <QDebug>
#include <stdexcept>
#include <cstring>

#pragma comment(lib, "ws2_32.lib")

Server::Server(QObject* parent) : QObject(parent), m_running(false),
m_serverSocket(INVALID_SOCKET) {
    WSADATA wsData;
    if (WSAStartup(MAKEWORD(2, 2), &wsData) != 0) throw
std::runtime_error("WSAStartup failed");

    m_serverSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (m_serverSocket == INVALID_SOCKET) throw std::runtime_error("Socket creation
failed");

    m_serverAddr.sin_family = AF_INET;
    m_serverAddr.sin_port = htons(54000);

```

```

m_serverAddr.sin_addr.s_addr = INADDR_ANY;

if (bind(m_serverSocket, reinterpret_cast<sockaddr*>(&m_serverAddr),
sizeof(m_serverAddr)) == SOCKET_ERROR)

    throw std::runtime_error("Bind failed");

if (listen(m_serverSocket, SOMAXCONN) == SOCKET_ERROR)

    throw std::runtime_error("Listen failed");

// FOR DB
g_dbManager.connect("WeatherStationDSN", "sa", "NewStrongPassword123!");
}

Server::~Server() {
    shutdown();
}

bool Server::start() {
    m_running = true;
    m_acceptThread = std::thread(&Server::acceptConnections, this);
    return true;
}

void Server::acceptConnections() {
    while (m_running) {
        sockaddr_in clientAddr;

        int clientSize = sizeof(clientAddr);

        SOCKET clientSocket = accept(m_serverSocket,
reinterpret_cast<sockaddr*>(&clientAddr), &clientSize);

        if (clientSocket == INVALID_SOCKET) continue;
    }
}

```

```

auto client = std::make_shared<ClientConnection>(clientSocket);
int socketId = static_cast<int>(clientSocket);

// Step 1: Send default sample time (2000 ms)
unsigned short defaultSampleTime = 2000;
QByteArray payload(reinterpret_cast<const char*>(&defaultSampleTime),
sizeof(defaultSampleTime));
unsigned short value;
std::memcpy(&value, payload.data(), sizeof(unsigned short));
unsigned short netValue = htons(value);
send(clientSocket, reinterpret_cast<const char*>(&netValue), sizeof(unsigned short), 0);

// Step 2: Receive first measurement
char buffer[20];
memset(buffer, 0, sizeof(buffer));

int ret = recv(clientSocket, buffer, sizeof(buffer), 0);
ClientData data;
std::memcpy(&data, buffer, sizeof(buffer));

// FOR DB
if(!g_dbManager.insertClient(socketId, std::to_string(socketId) + "_ClientFile.txt"))
{
    std::cout << "failure while saving client" << std::endl;
}

client->saveMeasurement(data, true);

// Step 3: sent OK response
client->sendDataQualityResponse(ret);

```

```

        // Add to map
        {
            std::lock_guard<std::mutex> lock(m_clientsMutex);
            m_clients[socketId] = client;
        }

        // start listenig
        emit clientConnected(socketId);
        std::thread(&Server::handleClient, this, client).detach();
    }
}

void Server::handleClient(std::shared_ptr<ClientConnection> client) {
    int socketId = static_cast<int>(client->socket());

    while (client->isConnected() && m_running) {
        auto result = client->receiveData();
        if (result.exitRequested) break;
    }

    {
        std::lock_guard<std::mutex> lock(m_clientsMutex);
        m_clients.erase(socketId);
    }

    emit clientDisconnected(socketId);
}

void Server::sendCommandToClient(int socketId, char commandId, const QByteArray&
payload) {
    std::lock_guard<std::mutex> lock(m_clientsMutex);

```

```

    if (m_clients.find(socketId) != m_clients.end()) {
        m_clients[socketId]->sendCommand(commandId, payload);
    }
}

```

```

void Server::shutdown() {
    m_running = false;

    {
        std::lock_guard<std::mutex> lock(m_clientsMutex);
        for (auto& [id, client] : m_clients) {
            client->close();
        }
        m_clients.clear();
    }
}

```

```

closesocket(m_serverSocket);
if (m_acceptThread.joinable())
    m_acceptThread.join();

emit serverStopped();
WSACleanup();
}

```

Приложение 3

```
#pragma once
```

```
#include <winsock2.h>
```

```
#include <string>
```

```
#include <fstream>
```

```

#include <atomic>
#include <mutex>
#include <iostream>
#include <QByteArray>

#include "DatabaseGlobal.hpp"
#include "ClientData.hpp"

struct ReceiveResult {
    bool exitRequested = false;
};

class ClientConnection {
public:
    explicit ClientConnection(SOCKET socket);
    ~ClientConnection();

    bool isConnected() const;
    SOCKET socket() const;

    ReceiveResult receiveData();
    void sendCommand(char commandId, const QByteArray& payload);
    void close();

    void saveMeasurement(const ClientData& data, bool isInitialData = false);
    void sendDataQualityResponse(int bytesReceived);

private:
    SOCKET m_socket;
    std::atomic<bool> m_connected;

```



```

std::ofstream m_outputFile;

std::string m_filename;

std::mutex m_fileMutex;

};

```

Приложение 4

```

#include "ClientConnection.hpp"

#include <cstring>

#include <winsock2.h>

```

```

#define BUFFER_SIZE 64

#define STRUCTURE_SIZE 20

#define FLAGS 0

#define RESPONSE_SIZE 1

```

```

ClientConnection::ClientConnection(SOCKET socket)
    : m_socket(socket), m_connected(true) {
    m_filename = "../client_data/" + std::to_string(socket) + "_ClientFile.txt";
}

```

```

ClientConnection::~ClientConnection() {
    close();
}

```

```

bool ClientConnection::isConnected() const {
    return m_connected;
}

```

```

SOCKET ClientConnection::socket() const {
    return m_socket;
}

```

```
}
```

```
void ClientConnection::sendCommand(char commandId, const QByteArray& payload) {  
    send(m_socket, &commandId, 1, 0);  
    if (!payload.isEmpty()) {  
        if ((commandId == '0' || commandId == '1') && payload.size() == sizeof(unsigned  
short)) {  
            unsigned short value;  
            std::memcpy(&value, payload.data(), sizeof(unsigned short));  
            unsigned short netValue = htons(value);  
            send(m_socket, reinterpret_cast<const char*>(&netValue), sizeof(unsigned short), 0);  
        } else {  
            send(m_socket, payload.data(), payload.size(), 0);  
        }  
    }  
}
```

```
void ClientConnection::close() {  
    if (m_connected) {  
        closesocket(m_socket);  
        m_connected = false;  
        if (m_outputFile.is_open())  
            m_outputFile.close();  
        std::remove(m_filename.c_str());  
    }  
}
```

```
void ClientConnection::saveMeasurement(const ClientData& data, bool isInitialData) {  
    std::lock_guard<std::mutex> lock(m_fileMutex);  
    m_outputFile.open(m_filename, std::ios::out | std::ios::app);  
    if (!m_outputFile.is_open()) return;
```

```

if(isInitialData) m_outputFile << "first measurement" << std::endl;
m_outputFile << data.latitude << "/" << data.longitude << "/"
    << data.aqi << "/" << data.temperature << std::endl;
if(isInitialData) m_outputFile << "======" << std::endl;
m_outputFile.close();

if(!g_dbManager.insertNotification(static_cast<int>(m_socket), data.latitude,
data.longitude, data.temperature, data.aqi))
{
    std::cout << "failure while saving notification" << std::endl;
}
}

void ClientConnection::sendDataQualityResponse(int bytesReceived){
    char response[RESPONSE_SIZE];
    if (bytesReceived < STRUCTURE_SIZE) {
        response[0] = '1';
    } else {
        response[0] = '0';
    }
    send(m_socket, response, sizeof(response), FLAGS);
}

ReceiveResult ClientConnection::receiveData() {
    ReceiveResult result;
    char buffer[BUFFER_SIZE] = {};

    int ret = recv(m_socket, buffer, sizeof(buffer), FLAGS);

    if (ret <= 0) {
        result.exitRequested = true;
    }
}

```

```

        return result;
    }

    std::string message(buffer, 0, ret);

    if (message == "exit") {
        result.exitRequested = true;
        return result;
    }
    if (message == "0") {
        return result;
    }
    if (message == "1") {
        result.exitRequested = true;
        return result;
    }
    if (ret == STRUCTURE_SIZE) {
        ClientData data;
        std::memcpy(&data, buffer, STRUCTURE_SIZE);
        saveMeasurement(data);
        return result;
    }

    {
        std::lock_guard<std::mutex> lock(m_fileMutex);
        m_outputFile.open(m_filename, std::ios::out | std::ios::app);
        if (!m_outputFile.is_open())
        {
            result.exitRequested = true;
            return result;
        }
    }

```

```

    }

    m_outputFile << "Requested data:" << std::endl;
    m_outputFile << std::string(buffer, 0, ret) << std::endl;
    send(m_socket, buffer, sizeof(buffer), 0);

    while (true) {
        memset(buffer, 0, sizeof(buffer));
        int bytesReceived = recv(m_socket, buffer, sizeof(buffer), 0);
        if (bytesReceived <= 0) break;

        if (std::string(buffer, 0, bytesReceived) == "end") break;

        m_outputFile << std::string(buffer, 0, bytesReceived) << std::endl;

        if(!g_dbManager.insertRequestedRecording(static_cast<int>(m_socket),
std::string(buffer, 0, bytesReceived)))
        {
            std::cout << "failure while saving requested data" << std::endl;
        }

        send(m_socket, buffer, sizeof(buffer), 0);
    }

    m_outputFile.close();
}

return result;
}

```

Приложение 5

```
#ifndef CLIENT_DATA_HPP
#define CLIENT_DATA_HPP
```

```
#pragma pack(push, 1)
```

```
struct ClientData {
    double latitude;
    double longitude;
    short aqi;
    short temperature;
};
```

```
#pragma pack(pop)
```

```
#endif
```

Приложение 6

```
// DatabaseGlobal.hpp
```

```
#pragma once
```

```
#include "DatabaseManager.hpp"
```

```
extern DatabaseManager g_dbManager;
```

Приложение 7

```
#include "DatabaseGlobal.hpp"
```

```
// Define the global instance
```

```
DatabaseManager g_dbManager;
```

Приложение 8

```
#pragma once
```

```
#include <windows.h>
```

```
#include <sql.h>
```

```
#include <sqlext.h>
```

```
#include <string>
```

```
#include <functional>
```

```
#include <sqltypes.h>
```

```
#include <mutex>
```

```
class DatabaseManager {
```

```
public:
```

```
    DatabaseManager();
```

```
    ~DatabaseManager();
```

```
    bool connect(const std::string& dsn, const std::string& user, const std::string& password);
```

```
    bool insertClient(int socketId, const std::string& filename);
```

```
    bool insertNotification(int socketId, double lat, double lon, int temp, int aqi);
```

```
    bool insertRequestedRecording(int socketId, const std::string& content);
```

```
    void clearAllData();
```

```
    std::vector<std::string> fetchClientData(int clientId);
```

```
    std::vector<std::pair<float, int>> fetchLastNotificationData(int clientId, int maxEntries);
```

```
private:
```

```
    SQLHENV m_env;
```

```

SQLHDBC m_dbc;

std::mutex m_mutex;

bool executePrepared(const std::string& query, const std::function<void(SQLHSTMT)>&
binder);

void printError(SQLSMALLINT handleType, SQLHANDLE handle);

};

```

Приложение 9

```
#pragma once
```

```

#include <QMainWindow>
#include <QtCharts/QChartView>
#include <QtCharts/QLineSeries>
#include <QtCharts/QValueAxis>
#include <QDebug>
#include <QTimer>

```

```
#include "Server.hpp"
```

```

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

```

```

class MainWindow : public QMainWindow {
    Q_OBJECT

```

```
public:
```

```

    MainWindow(QWidget* parent = nullptr);
    ~MainWindow();

```


private slots:

```
void onClientConnected(int socketId);  
void onClientDisconnected(int socketId);  
void onSetSampleTimeClicked();  
void onSetThresholdClicked();  
void onResetLogClicked();  
void onDownloadLogClicked();  
void onShutdownServer();  
void onStartServer();  
void onShowClientDataClicked();  
void onShowChartsClicked();
```

private:

```
Ui::MainWindow* ui;  
Server* m_server;  
QThread* m_serverThread;  
  
int getSelectedClientSocket();  
  
void setupCharts();  
void updateCharts();  
  
QLineSeries *m_tempSeries;  
QLineSeries *m_aqiSeries;  
QChart *m_tempChart;  
QChart *m_aqiChart;  
QChartView *m_tempChartView;  
QChartView *m_aqiChartView;  
QValueAxis *m_xAxisTemp;  
QValueAxis *m_yAxisTemp;
```

```

QValueAxis *m_xAxisAQI;
QValueAxis *m_yAxisAQI;

QTimer* m_chartUpdateTimer;
int m_currentChartClientId;
int m_chartUpdateCounter = 0; // Used as X-axis
};

```

Приложение 10

```

#include <QApplication>
#include "mainwindow.hpp"

int main(int argc, char *argv[]) {
    QApplication app(argc, argv);
    MainWindow w;
    w.show();
    return app.exec();
}

```

Приложение 11

```

cmake_minimum_required(VERSION 3.16)
project(WeatherStationServer)
set(CMAKE_CXX_STANDARD 17)
# Qt6
find_package(Qt6 REQUIRED COMPONENTS Widgets Charts)
set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTOUIC ON)
set(CMAKE_AUTORCC ON)
# Source files
file(GLOB_RECURSE SOURCES

```

```

    "controller/source/*.cpp"

    "gui/*.cpp"

    "database/*.cpp"

    "main.cpp"
)

file(GLOB_RECURSE HEADERS

    "controller/include/*.hpp"

    "gui/*.hpp"

    "database/*.hpp"
)

file(GLOB UI_FILES "gui/*.ui")

add_executable(ServerGUI ${SOURCES} ${HEADERS} ${UI_FILES})

# Include directories

target_include_directories(ServerGUI PRIVATE

    controller/include

    gui

    database
)

# Link libraries

target_link_libraries(ServerGUI PRIVATE

    Qt6::Widgets

    Qt6::Charts

    odbc32

    ws2_32

    crypt32

    advapi32

    userenv

    bcrypt
)

target_compile_definitions(ServerGUI PRIVATE NOMINMAX)

```